



Automated design of state transition rules in ant colony optimization by genetic programming: a comprehensive investigation

Bo-Cheng Lin¹ · Yi Mei¹ · Mengjie Zhang¹

Received: 19 May 2024 / Accepted: 5 January 2025 / Published online: 28 January 2025
© The Author(s) 2025

Abstract

The automated design of Ant Colony Optimization (ACO) algorithms has become increasingly significant, particularly in addressing complex combinatorial optimization problems. Although existing methods have achieved some success, they still face limitations, particularly the high dependency on expert knowledge, pre-solved data, and challenges in interpretability. Genetic Programming (GP), as a proven technology, has shown potential in optimizing the automated design state transition rules of ACO. However, existing research on GP-ACO is insufficient, particularly in terms of experimental validation and systematic evaluation. To address these issues, this study conducts comprehensive experiments to explore several key questions: the generality of GP-ACO on homogeneously distributed maps, the impact of different ACO variants on the learning capabilities of GP-ACO, the effect of 2-opt local search on the learning capabilities of GP-ACO, the enhancement of learning capabilities through the addition of more global information, and the interpretability of GP-ACO. The findings indicate that GP-ACO exhibits robust generality; variations among ACO variants have minimal impact on learning performance; 2-opt local search can somewhat diminish the performance of GP-ACO in the Max–Min Ant System; additional global information can significantly enhance the learning capabilities of GP-ACO; and GP-ACO has good interpretability.

Keywords Ant colony optimization (ACO) · Genetic programming (GP) · Combinatorial optimization · Travelling salesman problem · Hyper-heuristic

1 Introduction

The Ant Colony Optimization (ACO) algorithm [1], inspired by the pheromone-mediated foraging behavior observed in real ants, stands as one of the competitive methods for addressing complex and challenging combinatorial optimization problems [2]. Since its inception over three decades ago, extensive research and practical efforts have been dedicated to meticulously designing heuristic information for a variety of combinatorial optimization challenges [3]. Furthermore, these efforts have focused on how the parameter-

ized weights between heuristic information and pheromone trails can serve the probabilistic model of ACO [2]. By sampling this probabilistic model, ACO is capable of constructing optimal or near-optimal solutions within a finite timeframe. This probabilistic model, and the sampling thereof, is referred to as the state transition rules within the ACO framework [2]. Based on historical experience, the continuous improvement process of the pheromone trail is referred to as the pheromone update rule [4]. Together, the state transition rule and the pheromone update rule ensure the unique superiority of ACO. Currently, ACO achieves first-rate performance in numerous NP-hard combinatorial optimization problems [2], such as routing, scheduling, data mining, machine learning, and bioinformatics. Beyond academic inquiry, ACO also plays a pivotal role in practical applications [2]. For instance, companies like AntOptima¹ and Arcelor-Mittal² employ ACO to deliver solutions to customers and address real-world production challenges.

✉ Yi Mei
yi.mei@ecs.vuw.ac.nz

Bo-Cheng Lin
bocheng.lin@ecs.vuw.ac.nz

Mengjie Zhang
mengjie.zhang@ecs.vuw.ac.nz

¹ Centre for Data Science and Artificial Intelligence and School of Engineering and Computer Science, Victoria University of Wellington, PO BOX 600, Wellington, New Zealand

¹ www.antoptima.com/.

² corporate.arcelormittal.com/.

ACO has achieved significant success in addressing a variety of problems, largely due to the carefully designed heuristic information tailored for specific issues and the appropriate parameterization to balance exploration and exploitation [2]. These designs are predefined based on prior knowledge of the problems at hand, and researchers have exerted considerable effort to make ACO one of the most competitive methods for these specific challenges. However, as a meta-heuristic approach, in ACO designing suitable heuristic information and balancing the weights between heuristic information and pheromone trails for complex combinatorial optimization problems present substantial challenges. Moreover, due to the variable nature of combinatorial optimization problems, designing these components for each specific problem is nearly an impossible task. The diversity of combinatorial optimization issues and the extensive knowledge and experience required to design effective heuristics have become a bottleneck for those wishing to use ACO to solve combinatorial optimization problems.

Consequently, current researchers in ACO have begun exploring how to automate the design of ACO algorithms [5, 6]. Simultaneously, advancements in machine learning, particularly in deep learning and reinforcement learning over the past decade, have led to significant breakthroughs in fields such as image and natural language processing [7]. These methodologies are now being applied by researchers to assist meta-heuristic algorithms in solving combinatorial optimization problems [3], highlighting the advantages of ACO algorithms in this context. Furthermore, in recent years, the emergence of generative AI [8], led by large language models (LLMs), has introduced a burgeoning field focused on the automatic design of algorithms [9]. Given that LLMs possess foundational world knowledge, they can design algorithms independently of human input [10]. Thus, research on using large language models to craft algorithms for solving combinatorial optimization problems is increasingly prevalent [11]. This has led to the emergence a new category: automated ACO algorithm design.

Most existing studies have focused on the two core components of ACO framework: the state transition rule and the pheromone update rule. The purpose of the pheromone update rule is to ensure that components belonging to good solutions are more likely to be selected in the next iteration. A few studies have explored the automatic design of pheromone update rules [12, 13]. While some innovative patterns have been identified, the results have been suboptimal. In contrast, more successful studies have been conducted to the automatic design of state transition rules [2, 3, 5, 10]. The state transition rule is responsible for constructing solutions in ACO, and its effectiveness significantly impacts the quality of the final solution. Existing research results also indicate that the automatic design of state transition rules is a promising direction.

For using machine learning to aid the automatic design of state transition rules for ACO, most studies, particularly those based on neural networks, rely on predicting nodes [6] or heatmaps [3]. This approach can enhance heuristic information but requires a large amount of high-quality pre-solved solutions. Compared to manual design, prediction-based methods do not require expert experience, but they share similar drawbacks: for unseen combinatorial optimization problems, new algorithms need to be designed, or substantial pre-solved data is required to train the prediction model. When using LLMs to automatically design algorithms, neither human expert knowledge nor pre-solved data is necessary. However, current studies and tests of LLM-designed algorithms are primarily focused on prompt engineering [14–16]. Additionally, there is scant research on problems that are less explored or even entirely unseen by humans. This raises the question of whether LLMs can genuinely design algorithms autonomously or if they are merely summarizing human historical experiences to provide better designs. The answer to this question remains unknown.

Additionally, another drawback is that prediction-based algorithms, especially those based on neural networks, suffer from poor interpretability and are often viewed as black boxes [17]. On the other hand, LLMs can generate text to explain their behavior, which appears to offer interpretability. However, the safety and reliability of generative AI still require extensive research. Given the increasing demand for explainable artificial intelligence [18], it is evident that this drawback needs to be addressed.

The major problem to be solved in this paper concerns the manual formulation of state transition rules within ACO frameworks, which necessitates a substantial volume of domain-specific knowledge. This requirement often calls for extensive expertise that may not be readily available or could be prohibitively costly. Moreover, in the context of intricate problems or tasks, it is frequently beyond the capability of human experts to discern the nuanced interrelations among various elements or aspects of the variables or features involved. Conventionally, in manually crafted ACO algorithms, the structures of the state transition rules are rigid and confined—for instance, they often represent the product of the exponentials of pheromone and heuristic information. When addressing particularly complex issues, these limitations hinder experts from detecting subtle connections within the problem's elements, thereby impeding their ability to design optimally structured models. Conversely, the design of an effective ACO algorithm—and its associated state transition rules—predominantly relies on specific domain knowledge and experiential insights. To address this issue, we aim to use Genetic Programming (GP) [19], one of the most prominent methods in (evolutionary) machine learning, to automatically design the state transition rules in ACO. Genetic Programming has made significant advancements in

the field of automatic algorithm design, successfully tackling various complex combinatorial optimization problems [20]. GP algorithms do not rely on prior knowledge of the problem and offer excellent interpretability [18]. The GP searches within an extensive space of state transition rules, which includes the manually designed rules. This capability allows GP not only to autonomously derive models with robust structures and architectures but also to integrate manual rules and efficiently leverage domain-specific knowledge. Given the flexibility and comprehensiveness of the rule space, it is highly likely that there exist much more effective rules compared to the manually designed ones. Additionally, by training on a broad range of instances, we anticipate an iterative improvement in the state transition rules generation over successive generations.

Ranka et al. [5] is the only existing study to date to use GP for automatically designing state transition rules in ACO, which we refer to as GP-ACO. However, due to extensive computational time, their experiment was limited to training on only one instance and testing on two instances, which did not adequately assess the generality of GP-ACO. Additionally, their tests were confined to the Ant System (AS) [21] framework without exploring other ACO variants or incorporating local search techniques. Moreover, their implementation only utilized pheromone levels and the inverse of distances as terminals, without experimenting with a broader range of information as potential terminals. Building on these unresolved issues, our work aims to investigate the following critical questions in depth:

- Question 1: does GP-ACO exhibit generality when the distributions of training datasets and testing datasets are the same?
- Question 2: different ACO variants employ distinct designs for solution construction and pheromone update mechanisms, leading to unique characteristics for each ACO variant. How do these ACO variants influence the learning capabilities of GP-ACO?
- Question 3: Recent more advanced ACO designs have shown that incorporating local search can enhance the performance of ACO and handle the constraints well. On the other hand, it might reduce the influence of the starting-point generated by the state transition rule. Does the incorporation of local search weaken GP-ACO's learning effectiveness?
- Question 4: current state transition rule designs only utilize information from the current node. Would incorporating additional global information enhance GP-ACO's learning capability?
- Question 5: does GP-ACO remain effective when the distributions of training datasets and testing datasets are different?
- Question 6: how interpretable is GP-ACO?

By addressing these questions, we aim to deepen our understanding of GP-ACO and improve its effectiveness and generality in solving combinatorial optimization problems.

2 Related works

2.1 Design of ACO

The ACO algorithm [22], was initially developed to solve various path-finding/routing problems. Briefly speaking, a number of ants search for paths in an iterative process. In each iteration, each ant selects the next nodes based on the pheromone and heuristic information, which is known as the *state transition rule* [1]. After each iteration, the pheromone is updated, which can then guide the ants to find better paths in the subsequent iterations [23]. Central to the efficacy of ACO, derived from the path-finding behavior of ants, is the design of its state transition rule, which includes the structure and parameters such as the power of the pheromone and heuristic information [24]. This aspect is crucial as it directly influences the algorithm's performance in different problem contexts. Later, the algorithm was extended to more complex problems, such as the electric vehicle routing problem [25], dynamic scheduling Problems [26], and protein folding problems [27].

The effectiveness of ACO largely hinges on the quality of heuristic information design [2]. The design of heuristic information determines the extent to which ACO can utilize the settings and information embedded within the problem. Additionally, the balance between exploration and exploitation, pivotal in the design of state transition rules, is influenced by the parameters α and β . The magnitudes of these parameters dictate the ACO's preference for exploration versus exploitation within the probabilistic model formed by heuristic information and pheromone trails. Heuristic information, pheromone trails, and the parameters that balance their weights collectively constitute the state transition rules, which are the core components of ACO. Properly designing these elements is crucial as they define how effectively the algorithm can navigate and solve the optimization problem by leveraging both existing solutions (exploitation) and exploring new possibilities (exploration) [2]. The dynamic interplay of these factors ensures that ACO remains adaptable and efficient across various types of combinatorial optimization challenges.

2.2 Automated heuristic design

Automated heuristic design seeks to create methods that can automatically generate, select, and adapt heuristics to efficiently solve complex optimization problems. This approach aims to reduce the manual effort involved in crafting special-

ized heuristics for each unique problem, thereby increasing the flexibility and applicability of heuristic methods across diverse domains [28].

Hyper-heuristics represent an advanced category within automated heuristic design [29], distinguished by their ability to operate at a meta-level. Unlike traditional heuristics, which are directly applied to solve problems, hyper-heuristics focus on the selection and generation of heuristics based on their performance metrics during the optimization process [30]. This methodological shift allows hyper-heuristics to adapt dynamically to the problem at hand, selecting or creating the most effective strategies from a higher-level perspective [29].

In the domain of solving large-scale and dynamic combinatorial optimisation problems, such as job shop scheduling, hyper-heuristics have been successfully applied to sequence jobs on production lines [20]. Studies have shown that hyper-heuristics can outperform base-level heuristics by effectively selecting and combining simpler heuristics to adapt to the specific characteristics of production orders and machinery constraints [31]. Hyper-heuristics have also made significant inroads in solving arc routing problems, where the challenge is to efficiently determine the optimal path for traversing all required edges in a network while minimizing cost and adhering to constraints [32]. Timetabling is another area where hyper-heuristics have been extensively applied [33]. They are used to arrange courses, exam schedules, and room assignments in a way that minimizes scheduling conflicts and maximizes resource utilization. The adaptability of hyper-heuristics allows them to cope with varying institutional requirements and constraints [30].

GP represents a specialized branch of hyper-heuristics where the principles of evolutionary algorithms are used to generate new heuristics [29]. A GP hyper-heuristic (GPHH) evolves a population of heuristics, encoded as genetic programs, which are iteratively improved based on their performance in solving the given optimization problems [34]. GPHH have shown promising results, particularly in areas requiring highly customized solutions. For instance, they have been used to evolve decision rules and heuristics that can adapt to changes in problem parameters without human intervention [34]. This capability makes them highly valuable in dynamic environments such as real-time systems and online decision-making processes [20].

Hyper-heuristics, particularly those utilizing genetic programming [35], represent a cutting-edge approach in the field of automated heuristic design. They offer the dual benefits of flexibility and effectiveness, adapting to complex problem landscapes to generate customized and robust solutions [20]. As research progresses, the integration of reinforcement learning techniques is likely to further enhance the capabilities of GPHH [36], broadening their applicability and efficiency in solving dynamic optimization challenges.

2.3 Existing methods for automated design of ACO

In an effort to reduce reliance on manually designed rules, researchers have shifted their focus towards the automatic design of ACO algorithms. We review existing studies from three perspectives: firstly, what to automate in the design process; secondly, how to automate design, which involves model evaluation; and lastly, model representation.

Firstly, we address the scope of automated design. Current research includes efforts to automate the design of the ACO framework. Lopez-Ibanez et al. [37] deconstructed existing multi-objective ACO algorithms into components and devised a flexible framework for the automatic selection of these components. Aziz et al. [38] employed a Hyper-Heuristic approach to amalgamate multiple predefined base heuristics to augment ACO. Tiwari et al. [39] implemented automated control based on prior heuristics in ACO, enabling autonomous behavioral improvements. Subsequently, Tavares et al. [40] proposed employing Grammatical Evolution, a variant of GP, to evolve an ACO framework devoid of local search. Although this method was innovative and discovered some interesting structures, it failed to perform better than standard methods. Beyond framework studies, many researchers have begun to explore the automated design of pheromone update rules. Tavares et al. [12, 13] utilized GP and strongly typed GP to evolve the pheromone update strategies for ACO. While the evolved rules proved effective, they did not surpass the performance of standard methods. Studies focusing on the automated design of pheromone update rules are sparse, with more emphasis placed on the automated design of state transition rules—a highly promising direction. Notably, some studies have concentrated on the parameter combinations within the state transition rules. We consider these studies to lean more towards parameter tuning, and thus they are not extensively discussed within the scope of automated design. In pursuit of fully automating the design of state transition rules, Runka et al. [5] leveraged GP to evolve the state transition rules utilized by AS. Furthermore, Sun et al. [6] and Ye et al. [3, 10] have embarked on endeavors to automatically design the heuristic information in state transition rules. Although Runka et al. [5] completely redesigned the structure of the state transition rules, they retained the same terminal set as the manual rules and did not consider incorporating global information into the terminal set to better leverage the advantages of GP. On the other hand, Sun et al. [6] and Ye et al. [3, 10], while enhancing heuristic information through the use of trained predictive models to better utilize global information, still preserved the structure of the manually designed state transition rules unchanged. This approach significantly limits the flexibility compared to GP.

We then discuss the methodology of automation, also referred to as the evaluation approaches for models in auto-

ated design. Existing methods include simulation-based design, where models are assessed by running complete ACO simulations. This category predominantly employs hyper-heuristic approaches, as mentioned in earlier discussions regarding researchers like Lopez-Ibanez et al. [37], Tavares et al. [13, 40], and Runka et al. [5]. Algorithms automatically designed using methods like LLM are also evaluated similarly. With the rapid advancement of large language models [41], there is a growing interest in utilizing these models to design algorithms for combinatorial optimization problems [11, 42]. For example, Liu et al. [9] developed the LMEA algorithm, which leverages large language models as evolutionary optimizers. This innovative approach was initially tested on small-scale TSP problems. Chen et al. [16] introduced EvoPrompting for tackling neural architecture search problems, while Liu et al. [43] proposed the AEL algorithm, which is focused on algorithm design for solving TSP problems. Additionally, Liu et al. [15] applied the AEL method to automatically design a Guided Local Search algorithm for TSP challenges. Ye et al. [10] unveiled the ReEvo algorithm framework that integrates long-term reflection and short-term reflection with large language models to iteratively design algorithms. They employed GPT-3.5 to design heuristic measures for ACO algorithms targeting TSP problems, reporting performance that surpassed state-of-the-art neural heuristics, specifically DeepACO [3].

Simulation-based methods do not depend on pre-solved data, in contrast to supervised-learning methods. Sun et al. [6] introduced the ML-ACO method, which employs machine learning to learn features from edges of known optimal solutions, predicting whether a current edge will be part of an optimal solution in new problems. Beyond conventional machine-learning approaches, utilizing neural network-based methods to solve combinatorial optimization problems holds considerable promise and merits further exploration [44]. Liu et al. [45] developed the LN-ACO, enhancing traditional ant colony optimization by integrating an “intelligent ant” with a pre-trained convolutional neural network. Ye et al. [3] employed Graph Neural Networks (GNNs) [46] and reinforcement learning to refine heuristic measures within the state transition rules of ACO. Currently, the research using machine learning to optimize ACO algorithms is limited but possesses significant potential [47]. Although both ML-ACO and LN-ACO, along with DeepACO, are learning-based methods, ML-ACO functions primarily as a heuristic algorithm, whereas LN-ACO and DeepACO can be categorized as hyper-heuristic approaches. Simulation-based evaluations are more time-consuming as they necessitate a complete run of the ACO to assess the efficacy of the state transition rules, yet they do not depend on previously solved data. In contrast, supervised learning evaluations do not require a full ACO run but rely heavily on

pre-solved data to train the models, necessitating substantial effort in designing an effective loss function.

Finally, we discuss the representation of models in automated design. Current research can broadly be divided into several approaches. One common method employs GP’s tree structure to represent state transition rules or pheromone update rules, as seen in the studies by Runka et al. [5] and Tavares et al. [13]. Another approach utilizes Neural Network-based model representations, as demonstrated in the research by Ye et al. [3], and Liu et al. [45]. In addition to these, there are code-level representations, which involve directly generating or modifying algorithmic code, an approach also explored by Tavares et al. [40] and in studies based on LLMs [10, 15]. Both the tree structure and the code-level representation provide high interpretability. However, models based on neural networks typically suffer from poorer interpretability, posing challenges for understanding and modifying the underlying decision processes.

In considering the aspect of ‘what to learn’, the state transition rules hold substantial potential for optimization. Regarding ‘how to learn’, methods employing supervised learning utilize pre-solved data to train predictive models that enhance heuristic information. However, these predictive models often exhibit limited generalizability. In contrast, simulation-based approaches allow for continuous improvements within a complete ACO process, thereby offering superior generality. Finally, concerning ‘model representation’, predictive models based on neural networks suffer from poor interpretability. Conversely, tree-based and code-level methods provide enhanced interpretability. Upon comprehensive evaluation, it is posited that utilizing GP to automatically design ACO state transition rules effectively balances the advantages of these three dimensions.

3 GP for ACO

3.1 Framework of GP-ACO

The process of GP-ACO is illustrated in Fig. 1. The overall goal of GP-ACO is to design a new state transition rule for ACO automatically. GP-ACO consists of two main phases: training and test. The training phase begins with the initialization of a GP population. Each individual in this population represents a state transition rule. To evaluate a GP individual, it is integrated into the pre-specified ACO algorithm (e.g., Ant Colony System (ACS) or Max–Min Ant System (MMAS)) as its state transition rule. The resultant ACO algorithm is then applied to solve the training instances, and the quality of the obtained solutions will be set as the fitness of the individual. This fitness measure guides the selection process. At the end of the GP evolutionary process, the best GP individual (i.e.,

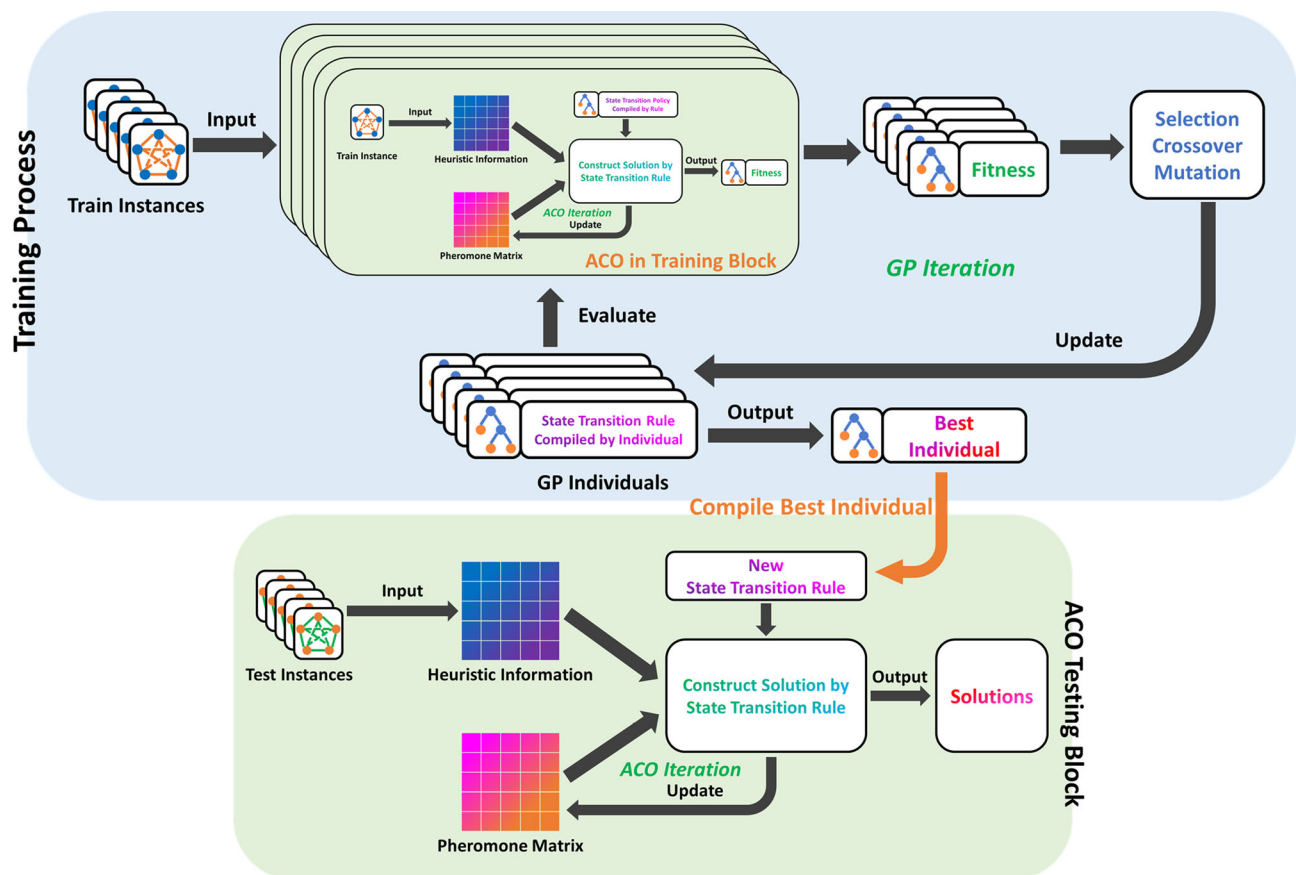


Fig. 1 Overall process of GP-ACO

state transition rule) is returned, and the corresponding ACO algorithm with the best state transition rule is thus learned.

In GP-ACO, we employed the subtree crossover method [19]. In this approach, a subtree is randomly chosen and exchanged between two parent trees, resulting in the formation of two new offspring trees. The point mutation [19] was applied to further introduce variability into the offspring. This mutation involves selecting a random node in the tree and altering its value, which can include changing a terminal node to another terminal or an internal node to a different function.

3.2 Solution construction

The solution construction in the GP-ACO method closely mirrors the procedure used in ACO. It begins with a random initial state (e.g., an empty route in the TSP) and then employs heuristics derived from the compilation of GP individuals as its state transition rule. This rule incorporates a roulette wheel selection for determining the next element to be added into the partial solution. In the AS and MMAS, as demonstrated, the priority value for each candidate element from the current element is calculated based on the levels of pheromone φ

and heuristic information η , followed by a roulette wheel selection process, which is given by (1). Conversely, in the ACS, as indicated in (2), there is a probability q_0 for using the ‘winner takes all’ strategy, meaning the candidate element with the highest priority value will be selected.

Specifically, taking the TSP as an example, a widely used state transition rule for AS algorithm is given by Equation (1), where the common parameter setting ($\alpha = 1$, $\beta = 5$) can be represented using GP’s tree structure shown in Fig. 2a. Similarly, the state transition rules for ACS and MMAS with their common parameter settings ($\alpha = 1$, $\beta = 2$) can also be represented using tree structures depicted in Fig. 2b. The commonly used state transition rule in ACS is given by Equation (2). For each candidate to be added during the solution construction process, a priority value is calculated using the state transition rule, normalized, and then the next element is selected using the roulette wheel method. Therefore, as shown in Fig. 2, φ and d can be considered as two terminals in the GP tree, with the introduction of more operators as part of the function set, allowing GP to evolve and learn new better state transition rule for calculating the priority value of each candidate.

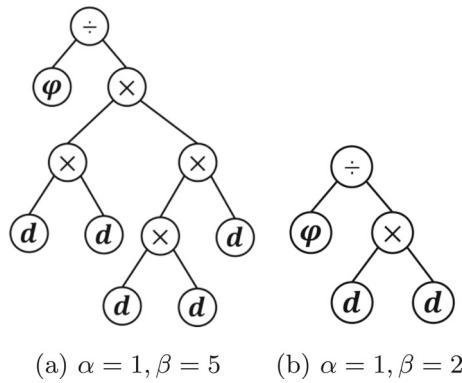


Fig. 2 Representing State Transition rule in Tree Structure

$$p_{ij} = \begin{cases} \frac{\varphi_{ij}^\alpha \eta_{ij}^\beta}{\sum_{l \in \text{feasible solution component}} \varphi_{il}^\alpha \eta_{il}^\beta}, & \text{if } l \in \text{feasible solution component} \\ 0, & \text{otherwise} \end{cases} \quad (1)$$

$$j = \begin{cases} \arg \max_{j \in \text{feasible solution component}} (\varphi_{ij}^\alpha \eta_{ij}^\beta), & \text{if } q \leq q_0 \\ \text{roulette wheel}, & \text{otherwise} \end{cases} \quad (2)$$

3.3 Fitness evaluation

As illustrated in Algorithm 1, the method for fitness evaluation of each individual in the GP training process involves the following steps: Each individual is first compiled into a state transition rule, which then replaces the original state transition rule in ACO. Next, a training instance is loaded, and a complete ACO process is executed to solve for the best solution of that instance. The objective value of the obtained solution (e.g., length of the TSP tour) is used as the fitness value of the individual. Additionally, we apply the mini-batch strategy (changing instance each generation). In other words, in the i th generation, the fitness of all individuals is assessed based on the i th training instance. Although ACO is a stochastic algorithm, we still use the setting that a single run with an arbitrary random seed is applied here to speed up training.

4 Experimental setup

To address the research questions 1-5 posed in Sect. 1, we utilize the TSP as a case study and design the following experiments:

- Experiment 1: to answer Question 1 and Question 2, we apply the GP-ACO framework to AS and ACS and

Algorithm 1 Training Process of GP-ACO

Require: training_instances
Ensure: best_individual

```

1: load training_instances, initialize GP population;
2: for  $i = 0$  to  $max\_iteration$  do
3:   instance = training_instance[i];
4:   for  $n = 0$  to  $pop\_size$  do
5:     calculate fitness for each individual based on Sect. 3.3;
6:      $n++$ ;
7:   end for
8:   generate offspring by parent: selection, crossover, mutation;
9:    $i++$ ;
10: end for
11: return best_individual;

```

MMAS, without considering local search. Train and test on three scales of TSP problems (TSP20, TSP20-100, TSP100) and compare the results with the baseline ACO algorithm and the LLM-designed ACO algorithm (the best heuristic information reported in the ReEvo algorithm). For clarity, we denote LLM-designed ACO as LLM-ACO.

- Experiment 2: to answer Question 3, we introduce *localsearch* to all the ACO variants in Experiment 1 while keeping the training and testing conditions unchanged, and compare their performance.
- Experiment 3: to answer Question 4, building on Experiments 1 and 2, add four terminals containing global information to GP-ACO, which we refer to as xGP-ACO, while maintaining the same training and testing conditions. Compare xGP-ACO with ACO baseline, LLM-ACO and GP-ACO to verify the impact of the new global terminals to learning ability of GP-ACO.
- Experiment 4: to answer Question 5, we directly use the state transition rules trained by xGP-ACO on the TSP100 dataset from Experiment 3, test on the publicly available TSPLIB dataset.

4.1 Datasets

The TSP is a widely studied combinatorial optimization problem [48]. It involves a salesman who must traverse a network of cities, visiting each exactly once and returning to the origin point. Formally, given a complete graph $G = (V, E)$ with vertices V representing cities and edges E representing the paths between them, each edge $(i, j) \in E$ is assigned a weight $w(i, j)$, which reflects the cost or distance between cities i and j . The objective is to find the shortest possible route that visits each city exactly once and returns to the starting point, minimizing the total path cost.

For Experiments 1, 2, and 3, we selected three datasets for the TSP in three different sizes:³ datasets with 20 nodes, 100

³ We will provide the datasets on Github, if the paper is accepted.

nodes, and a mixed dataset with nodes varying from 20 to 100. Each dataset consists of a training set with 50 instances and a test set with 10 instances. Notably, in the mixed dataset with 20-100 nodes, the instances contain a random number of nodes varying between 20 and 100. Each instance is generated using the portgen generator, which was previously used to create testbeds for the 8th DIMACS Implementation Challenge [49]. The portgen generator produces TSP instances, known as *rue* instances, by uniformly and randomly placing points on a two-dimensional plane. For Experiment 4, we utilized 15 instances from the publicly available TSPLIB dataset as the test datasets.

Using *rue* TSP instances as training datasets is a widely adopted approach in the research on the automated design of ACO [3, 9, 10]. This method is particularly relevant as we aim to compare GP-ACO with LLM-designed ACO reported by Ye et al. [10]. To ensure a fair comparison, it is necessary to select TSP instances from the same distribution for our training datasets. Furthermore, in Experiment 4, we employed the TSPLIB dataset, which comprises instances of varying scales and distributions. TSPLIB is a well-known benchmark dataset in TSP [50], allowing for a comprehensive evaluation of algorithm performance by comparing the results against known optimal solutions.

4.2 Algorithm settings

For the ACO algorithms, the population size was set to 20 for instances with 20 nodes and 50 for instances with more than 20 nodes, with the number of iterations fixed at 100 generations. For all the ACO algorithms we used the commonly adopted parameter settings: for the AS algorithm $\alpha = 1$, $\beta = 5$, $\rho = 0.9$ [21]; for the ACS algorithm, the parameters are set as $\alpha = 1$, $\beta = 2$, $\rho = 0.9$, $q_0 = 0.9$ [1]; and for the MMAS algorithm, the parameters are $\alpha = 1$, $\beta = 2$, $\rho = 0.98$ [4]. For GP-ACO, we used the same values for ρ and q_0 as that used in the corresponding ACO framework. The same parameters were used for both training and testing. The pheromone update mechanism adheres to the designs established in AS [21], ACS [1], and MMAS [4]. For the 2-opt local search, the total number of steps is set to one-fourth of the number of cities in the instance.

For GP, the population size was set at 100, with an evolutionary span of 50 generations. The crossover rate was set to 0.8, and the mutation rate was 0.15, and the reproduction rate was 0.05. The number of elite individuals was 10, tournament selection was used with a tournament size of 4, and the maximum tree depth was set at 5. The terminal set included two terminals: pheromone concentration φ and distance d . The function set comprised five operations: addition, subtraction, multiplication, protected division (returning 1 when divided by 0), and negation.

All algorithms were implemented in Python. Each compared ACO algorithm was independently run 30 times for every test instance. To ensure statistical significance, the GP-ACO algorithm was independently trained 30 times, returning 30 learned state transition rules. Each of the 30 learned rule was then embedded into the ACO algorithm and independently tested 30 times on each test instance.

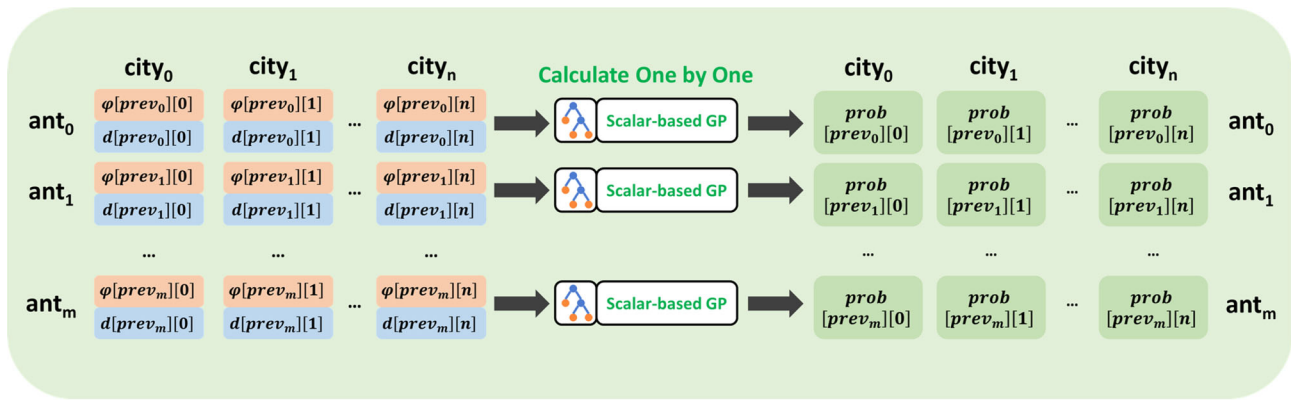
4.3 Matrix-based GP-ACO to accelerate training process

Since both GP and ACO are population-based algorithms that require iterative cycles to find the best solutions, we find that the training process of GP-ACO involves nesting the ACO iterative framework within the GP iterative framework. This evidently results in significant computational time. Ranka et al. [5] also reported the issue of prolonged computation time in their work. To address the computational time issue, we designed a matrix-based GP-ACO, which replaces scalar operations in the original state transition rules with matrix operations.

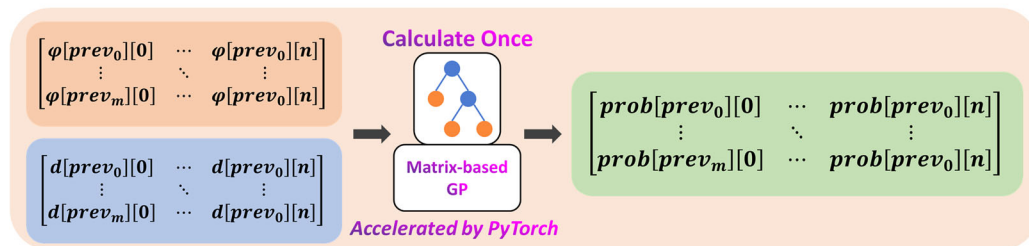
As illustrated in Fig. 3a, scalar-based GP-ACO requires a calculation to be performed for each ant and every candidate city during the application of the state transition rules. Conversely, matrix-based GP-ACO restructures the computation process and employs PyTorch-accelerated matrix operations to replace basic scalar operators, as shown in Fig. 3b. This approach leverages the efficiency of matrix-based computations to enhance the performance and scalability of the GP-ACO algorithm.

Simultaneously, matrix-based GP-ACO adopts a different parallel structure compared to scalar-based GP-ACO, as depicted in Fig. 4. Taking the example of utilizing a 16-core CPU, scalar-based GP-ACO assigns 16 threads, each of which calls upon a single CPU core during the ACO process. In contrast, matrix-based GP-ACO leverages PyTorch-accelerated matrix operations, allowing for the utilization of multiple cores to speed up matrix computations during the ACO process. As shown in Fig. 4b, we can allocate 4 main threads, with each main thread further deploying 4 cores for PyTorch acceleration. This architecture significantly enhances the computational efficiency and performance of the ACO algorithm by optimizing resource usage across multiple CPU cores. On an Intel i7-13700 CPU, using the CPU version of PyTorch and leveraging 16 cores, the training time of matrix-based GP-ACO can be reduced by approximately 80% than scalar-based GP-ACO.⁴

⁴ We can provide the codes on Github, if the paper is accepted.



(a) Scalar-based GP-ACO state transition rule calculation.



(b) Matrix-based GP-ACO state transition rule calculation.

Fig. 3 Scalar-based GP-ACO and Matrix-based GP-ACO state transition rule calculation

5 Results and discussions

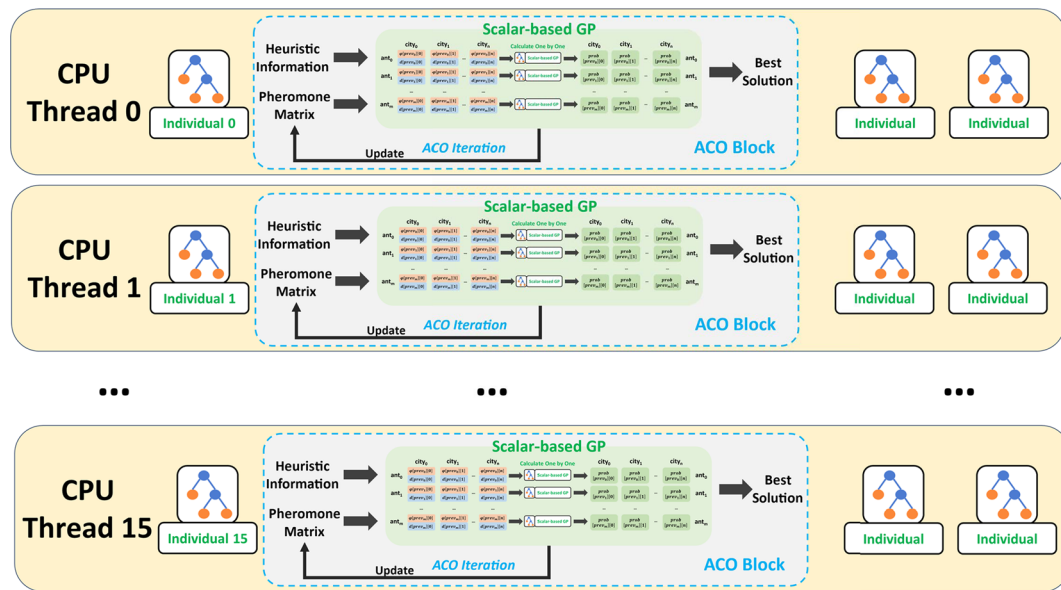
5.1 Experiment 1: generality of GP-ACO

To reduce the bias associated with different scales of instances, we normalized all results by dividing the costs for each instance by the best cost obtained in the ACO baseline, LLM-ACO, and GP-ACO for that instance. The normalized results, categorized according to the scale of the TSP problem, were then compared with the outcomes of the ACO algorithm using the Wilcoxon rank-sum test at a significance level of 0.05. For clarity, we have designated the GP-ACO using the AS framework as GP-AS, and likewise, the other two are named GP-ACS and GP-MMAS. The LLM-designed ACO algorithms are correspondingly named LLM-AS, LLM-ACS, and LLM-MMAS. Table 1 illustrates the mean, standard deviation, minimum, and maximum costs found by the ACO and GP-ACO algorithms across the three problem scales. All values are rounded to four decimal places. Notably, because GP-ACO was independently trained 30 times, its standard deviation is calculated from the 30 independently test results. Bold indicates the best mean within the same ACO variant, while bolditalics indicates the best mean across all ACO variants. A ‘+’ sign indicates a significant superiority of the GP-ACO over the corresponding ACO

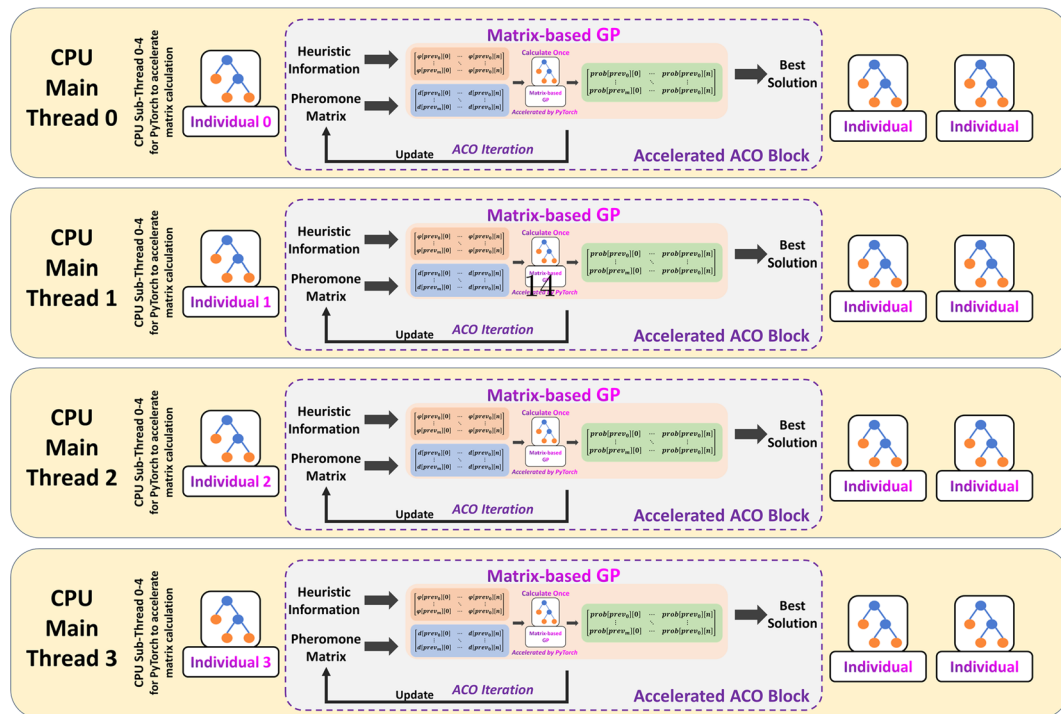
algorithm, a ‘–’ signifies an inferior performance, and an ‘=’ denotes no significant difference.

As shown in the results in Table 1, GP-AS and GP-ACS consistently find better solutions than both the baseline ACO and LLM-ACO across the three scales of TSP problems. While GP-MMAS demonstrates a significant improvement over the MMAS baseline, it remains closely competitive with LLM-MMAS. Observing the standard deviations, we can see that the ACO algorithms exhibit high stability, while the GP-ACO standard deviations are generally non-zero but still very low. These slight variations are due to the differences in performance of the state transition rules obtained from 30 independent training runs. When comparing the ability to find better solutions, we focus on the minimum values. On the TSP20 dataset, there is no difference, as all algorithms reach the lower bound. For the TSP20-100 dataset, the ACS baseline and LLM-ACS do not reach the lower bound, whereas GP-ACS does. On the TSP100 dataset, only LLM-MMAS reaches the lower bound. Although GP-AS and GP-ACS do not achieve the lower bound, they still perform better than the baseline ACO.

To visualize the results more comprehensively, box plots of the 30 ACO runs with the state transition rules of the original ACO and learned by the 30 GP runs were shown for analysis. Figure 5 were categorized by problem scale and ACO variants, with the horizontal axis representing the ACO



(a) Scalar-based GP-ACO parallel structure.



(b) Matrix-based GP-ACO parallel structure.

Fig. 4 Comparison of GP-ACO parallel structures

algorithms and the 30 randomly seeded GP-ACO, and the vertical axis showing distributions of the normalized solution quality (the smaller the better) of the corresponding 30 ACO runs. The orange boxplots denote GP-ACO significantly outperformed the original ACO, and green filling is the opposite. The grey boxplots mean there is no statistical

difference from the original ACO. Observations from Fig. 5 indicate that focusing on the count of orange and green colors is sufficient.

Upon observing the box plots in Fig. 5, we can see that, except for the TSP20-100 scenario where GP-AS and GP-ACS occasionally exhibit 1-2 instances of inferior perfor-

Table 1 The normalized costs of GP-ACO, ACO baselines, and LLM-ACO on the test datasets

TSP		AS			ACS			MMAS		
		Baseline	LLM	GP	Baseline	LLM	GP	Baseline	LLM	GP
20	Mean	1.0028 +	1.0020 +	1.0019	1.0063 +	1.0074 +	1.0025	1.0164 +	1.0031 –	1.0039
	Std	0	0	0.0001	0	0	0.0002	0	0	0.0002
	Min	1	1	1	1	1	1	1	1	1
	Max	1.0217	1.0218	1.0201	1.0359	1.0604	1.0172	1.1327	1.0186	1.0467
20–100	Mean	1.0364 +	1.0271 +	1.0265	1.0499 +	1.0418 +	1.0306	1.0632 +	1.0076 –	1.0254
	Std	0	0	0.0004	0	0	0.0004	0	0	0.0001
	Min	1	1	1	1.0006	1.0016	1	1	1	1
	Max	1.1042	1.0747	1.3813	1.1445	1.0831	1.1660	1.2101	1.0440	1.1212
100	Mean	1.0784 +	1.0533 +	1.0105	1.0867 +	1.0721 +	1.0446	1.1248 +	1.0144 –	1.0400
	Std	0	0	0.0001	0	0	0.0005	0	0	0.0001
	Min	1.0359	1.0127	1.0005	1.0310	1.0148	1.0162	1.0481	1	1.0071
	Max	1.1346	1.0962	1.0274	1.1451	1.1154	1.0799	1.2604	1.0680	1.0852

mance compared to the baseline ACO, GP-ACO consistently outperforms the baseline ACO in nearly all other cases.

Unlike GP-AS and GP-MMAS, GP-ACS introduces a new parameter $q_0=0.9$ in the path construction phase. This implies a 90% probability of selecting the next node directly based on the highest prior value. As a result, GP-ACS predominantly relies on the state transition rules discovered by GP-ACS, with a 90% likelihood of greedily choosing the maximum value of the state transition rule for the next node. Observationally, the outcomes of GP-ACS are significantly better than those of ACS baseline, indicating the effectiveness of our GP-ACS algorithm.

In summary, we now address the Question 1 and the Question 2:

- Answer 1: The GP-ACO demonstrates generality. Across three scales of the TSP problem, the trained GP-ACO consistently outperforms the ACO baseline, and this advantage becomes more pronounced as the problem size increases.
- Answer 2: Different ACO frameworks do not affect the learning capability of the GP-ACO. In three variants of the ACO, the GP-ACO significantly surpasses the ACO baseline in performance. Additionally, we discovered that the state transition rules designed in GP-AS and GP-ACS notably outperform those in LLM-ACO, whereas the reverse is true for GP-MMAS.

5.2 Experiment 2: GP-ACO+2-opt

Local search is crucial for the ACO algorithm as it enhances the quality of solutions and utilizes superior solutions to update the pheromone levels during both local and global updates. However, incorporating local search may reduce

the ACO's reliance on its state transition rules, and it is unknown whether this will affect the learning capabilities of the GP-ACO. To investigate this issue, we incorporated the 2-opt algorithm—a widely recognized and highly effective classic local search technique—throughout the training and testing phases of Experiment One. For consistency and ease of comparison, the same normalization methods from the initial experiment were employed. The results are presented in Table 2 as shown.

As demonstrated in Table 2, GP-AS+2-opt and GP-ACS+2-opt consistently outperform the ACO+2-opt baseline in all three scenarios, either matching or surpassing the lower bounds. Particularly in the TSP20-100 and TSP100 cases, they exceed the performance of LLM-ACO. However, GP-MMAS+2-opt does not surpass the MMAS+2-opt baseline in the TSP20-100 scenario. Compared to the results in Experiment One, the introduction of the 2-opt algorithm has significantly enhanced the stability of all algorithms, with each method consistently finding solutions at or near the lower bounds. In the context of ACS and MMAS, both benefit from the global pheromone update strategies enhanced by superior solutions provided through local search. However, ACS relies more heavily on state transition rules than MMAS, which is why GP-ACS+2-opt shows a more marked improvement over the ACS+2-opt baseline, whereas the enhancement from GP-MMAS+2-opt over the MMAS+2-opt baseline is minimal.

To better observe the performance of GP-ACO+2-opt across 30 independent training sessions, we also constructed box plots, as illustrated in the Fig. 6. GP-AS+2-opt and GP-ACS+2-opt significantly outperform the ACO+2-opt baseline in most scenarios. On the other hand, GP-MMAS+2-opt shows commendable performance in the TSP100 case but

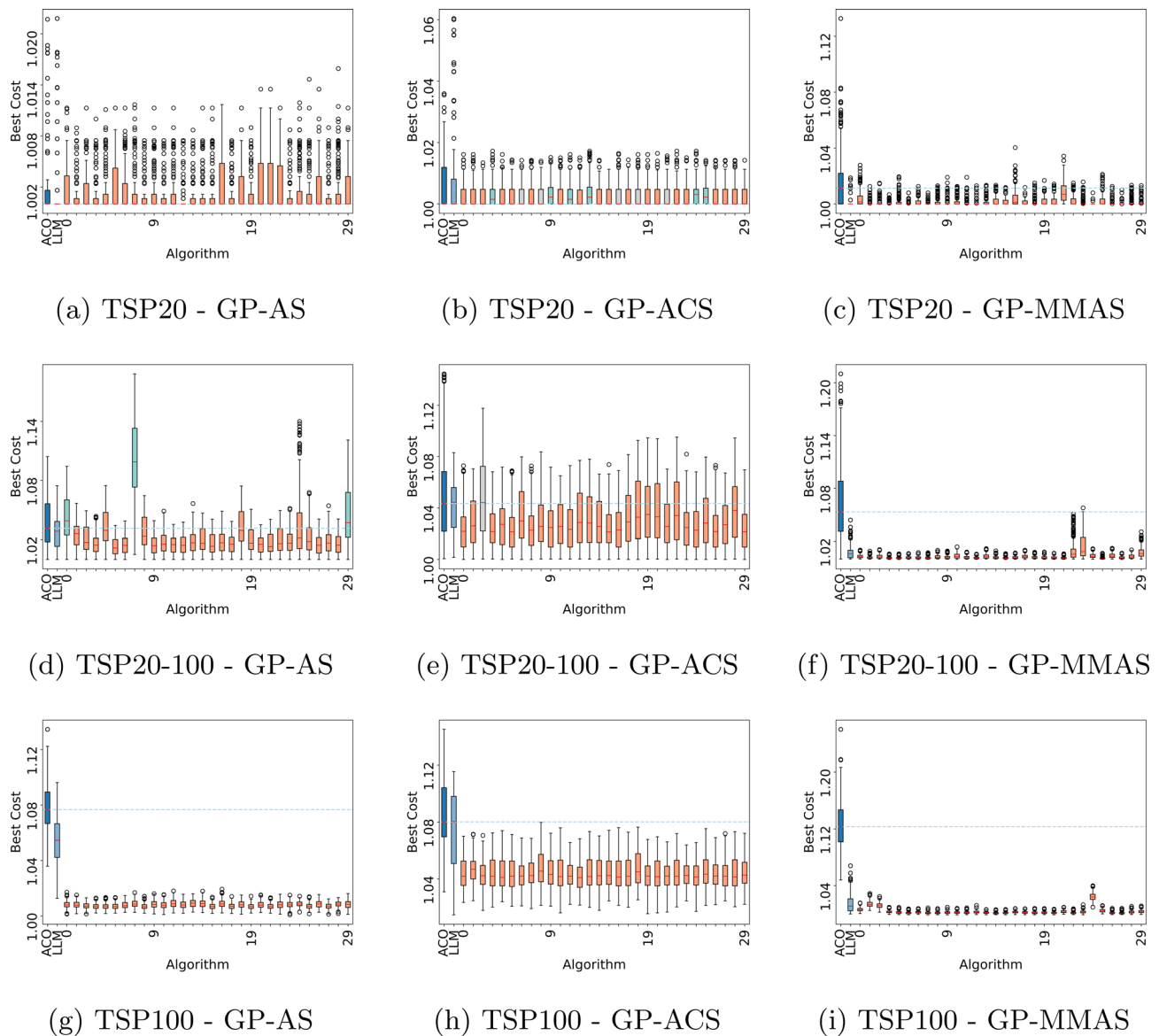


Fig. 5 Boxplot of ACO & the state transition rules obtained by GP-ACO in the 30 GP runs

exhibits minimal differences or even deterioration compared to the baseline in the TSP20-100 scenario.

In summary, we can address the Question 3 and provide additional insights into the Question 1 and Question 2:

- Answer 3: The introduction of local search does not impact the learning effectiveness of GP-ACO in the GP-AS+2-opt and GP-ACS+2-opt scenarios, but it diminishes the learning capabilities of GP-ACO in the GP-MMAS+2-opt scenario.
- Enhancing Answer 1: GP-ACO+2-opt continues to exhibit generality. However, in the case of GP-MMAS+2-opt, its performance in generality did not meet expectations.
- Enhancing Answer 2: ACO variants that rely more on state transition rules are almost unaffected by the introduction of 2-opt local search regarding the learning capabilities of GP-ACO. Conversely, ACO variants that depend more heavily on pheromone updates show a reduction in GP-ACO's learning abilities when local search is implemented.

5.3 Experiment 3: xGP-ACO and xGP-ACO+2-opt

In the previous experiments, we investigated scenarios where only two terminals were used as inputs for GP-ACO. In most cases, GP-ACO outperformed ACO. Consequently, in this experiment, we explore the effectiveness of including more

Table 2 The normalized costs of GP-ACO+2-opt, ACO+2-opt baselines, and LLM-ACO+2-opt on the test datasets

TSP		AS+2-opt			ACS+2-opt			MMAS+2-opt		
		Baseline	LLM	GP	Baseline	LLM	GP	Baseline	LLM	GP
20	Mean	1 =	1 =	1	1.0003 +	1 -	1	1.0002 +	1 -	1.0001
	Std	0	0	0	0	0	0	0	0	0
	Min	1	1	1	1	1	1	1	1	1
	Max	1	1	1	1.0186	1.0000	1.0186	1.0186	1	1.0090
20–100	Mean	1.0018 +	1.0037 +	1.0008	1.0021 +	1.0046 +	1.0008	1.0007 -	1.0008 +	1.0008
	std	0	0	0	0	0	0.0001	0	0	0.0001
	Min	1	1	1	1	1	1	1	1	1
	Max	1.0101	1.0162	1.0452	1.0097	1.0173	1.0123	1.0102	1.0084	1.0495
100	Mean	1.0045 +	1.0091 +	1.0021	1.0045 +	1.0096 +	1.0017	1.0017 =	1.0011 -	1.0013
	Std	0	0	0.0001	0	0	0.0001	0	0	0.0001
	Min	1	1	1	1	1	1	1	1	1
	Max	1.0184	1.0269	1.0136	1.0211	1.0281	1.0117	1.0112	1.0185	1.0276

terminals as inputs for GP-ACO. We anticipate that additional terminals will allow the state transition rules to access more global information, thereby identifying improved state transition rules. Building on the original distance (d) and pheromone concentration (φ) inputs, we included four more terminals: average distance to all the candidate nodes from the current node ($\bar{d}$), average pheromone concentration to all the candidate nodes from the current node ($\bar{\varphi}$), the total number of nodes (n), and the number of candidate nodes (n_u). All other parameters remain unchanged from the previous section. The GP-ACO variant with six terminal inputs is named xGP-ACO.

To ensure consistency in the results, we employed the same lower bound settings for normalization as in the previous section. Therefore, if xGP-ACO finds a better solution, its normalized value will be less than 1.

The results of xGP-ACO are displayed in Table 3. Without the introduction of local search, xGP-ACO significantly outperforms GP-ACO, the ACO baseline, and LLM-ACO. Notably, xGP-MMAS achieves not only the global optimum mean in both TSP20 and TSP100 scenarios but also finds solutions that surpass the existing lower bounds.

Upon the introduction of local search, the results of xGP-ACO+2-opt are as shown in Table 4. xGP-AS+2-opt performs excellently, significantly surpassing GP-ACO, the ACO baseline, and LLM-ACO in both TSP20-100 and TSP100 scenarios, even achieving the global optimum mean in TSP20-100. xGP-ACS+2-opt is slightly inferior only to GP-ACS+2-opt in the TSP100 scenario, but it significantly outperforms other algorithms in all other conditions. The performance of xGP-MMAS+2-opt, however, is not as favorable; it only manages to outperform the ACO+2-opt baseline,

and does not exceed the performance of GP-ACO+2-opt or LLM-ACO+2-opt in other scenarios.

The inclusion of a greater number of terminals means a larger search space. Without local search, the ACO algorithm relies heavily on its state transition rules, and environmental pressures make xGP-ACO more effective in exploring this expanded search space. However, the introduction of local search decreases the dependency of the ACO algorithm on state transition rules, reducing environmental pressures. Consequently, xGP-ACO+2-opt's exploration of the search space is quite limited, a phenomenon that is particularly evident in the case of xGP-MMAS+2-opt. This reduced exploration can be attributed to the diminished influence of state transitions, which are crucial for effectively navigating broader solution landscapes when enhanced by local search strategies.

Thus, we can address the Question 4 and further supplement our responses to the Question 2 and Question 3:

- Answer 4: The introduction of additional global information in xGP-ACO without local search proves highly effective, significantly enhancing learning capabilities compared to GP-ACO. With local search, incorporating more global information significantly improves the learning capabilities of GP-AS+2-opt but weakens those of xGP-MMAS+2-opt, which has less dependency on state transition rules.
- Enhancing Answer 2 and Answer 3: After introducing more global information, and excluding the consideration of local search, variations in ACO frameworks do not reduce the learning capabilities of xGP-ACO. However, when the local search is introduced, the lesser dependence on state transition rules in MMAS variants diminishes the learning capabilities of xGP-ACO+2-opt.

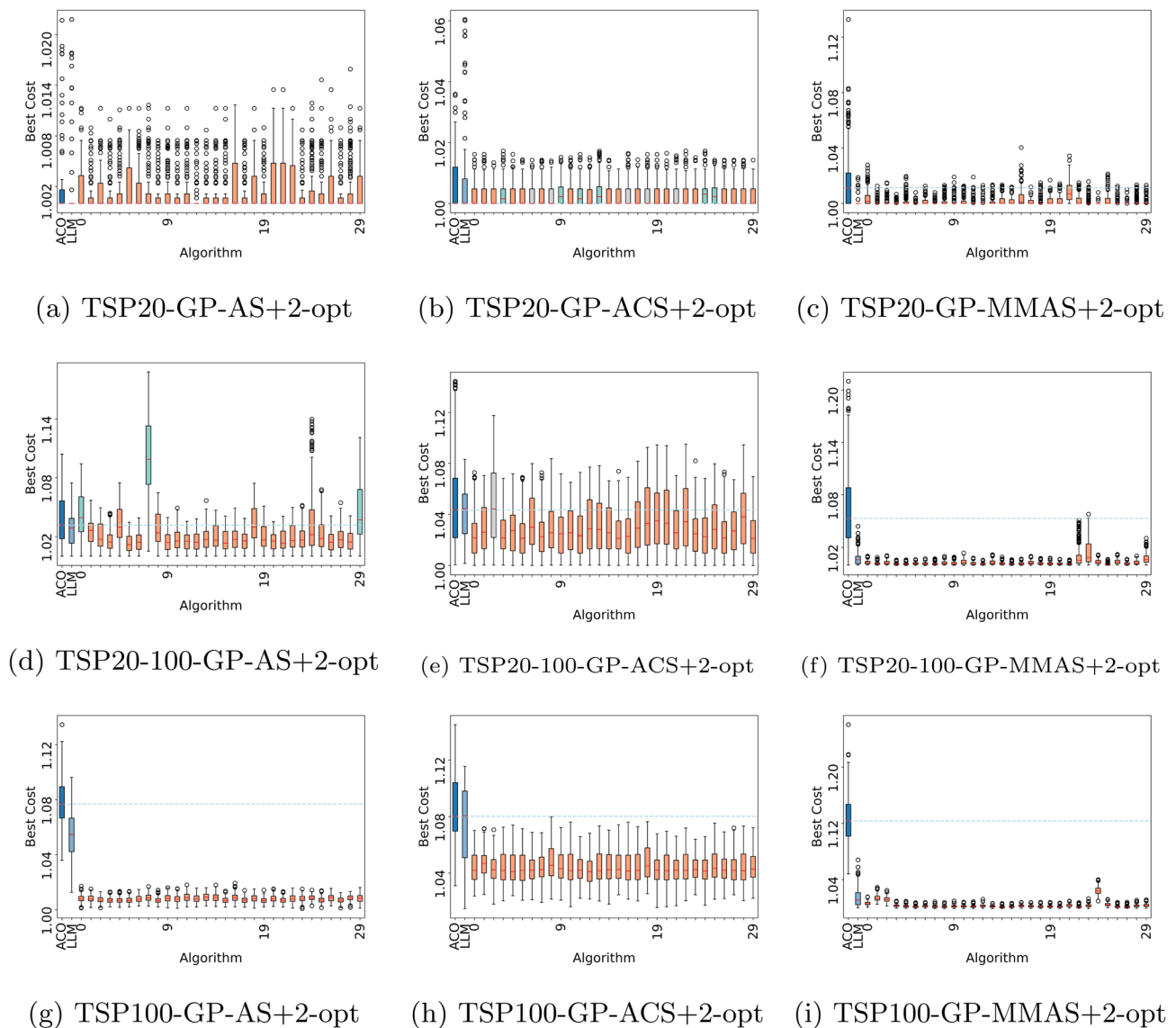


Fig. 6 Boxplot of ACO+2-opt & the state transition rules obtained by GP-ACO+2-opt in the 30 GP runs

5.4 Experiment 4: xGP-ACO and xGP-ACO+2-opt on TSPLIB

The three experiments previously conducted have validated the generality of the state transition rules derived from GP-ACO and xGP-ACO for problems distributed in the same manner. Experiment 3 demonstrated the superior performance of xGP-ACO, leading us to select it as our test subject for this experiment. We utilized results trained on TSP100 to test across 15 instances from the publicly available TSPLIB dataset. Given that xGP-ACO was trained independently 30 times, we also selected the best state transition rule from these training sessions, referred to as xGP-best. It is important to note that finding optimal solutions often requires adjusting the parameters of ACO and 2-opt as problem sizes

increase, to evaluate more solutions and identify superior ones. Our experiment aims to assess the performance of the xGP-ACO framework on larger scales and on instances with different geographical distributions, not to demonstrate that xGP-ACO can find the optimal solution for each instance. Therefore, for consistency and ease of comparison, we set the same algorithmic parameters for all instances. For comparative purposes, we focused on the average gap% between the solutions obtained and the known optimal solutions for each instance, with results displayed in the Tables 5 and 6 as shown.

We first compared the differences within the same ACO variant. For AS and xGP-AS, xGP significantly reduced the gap between the baseline and the known optimal solution, from 13.51 to 7.43%. The heuristic measures designed by

Table 3 The normalized results of xGP-ACO, ACO baselines, and LLM-ACO on the test datasets

TSP	AS				ACS				MMAS			
	Baseline	LLM	GP	xGP	Baseline	LLM	GP	xGP	Baseline	LLM	GP	xGP
20	Mean	1.0028 +	1.0020 +	1.0019 +	1.0014	1.0063 +	1.0074 +	1.0025 +	1.0164 +	1.0031 +	1.0039 +	1.0015
	Std	0	0	0.0001	0.0001	0	0	0.0002	0	0	0.0002	0.0002
	Min	1	1	1	1	1	1	1	1	1	1	1
	Max	1.0217	1.0218	1.0201	1.0159	1.0359	1.0604	1.0172	1.1327	1.0186	1.0467	1.0403
20–100	Mean	1.0364 +	1.0271 +	1.0265 +	1.0235	1.0499 +	1.0418 +	1.0306 +	1.0632 +	1.0076 +	1.0254 +	1.0029
	Std	0	0	0.0004	0.0004	0	0	0.0004	0	0	0.0001	0.0001
	Min	1	1	1	1	1.0006	1.0016	1	1	1	1	0.9940
	Max	1.1042	1.0747	1.3813	1.1882	1.1445	1.0831	1.1660	1.2101	1.0440	1.1212	1.0579
100	Mean	1.0784 +	1.0533 +	1.0105 +	1.0079	1.0867 +	1.0721 +	1.0446 +	1.1248 +	1.0144 +	1.0400 +	1.0039
	Std	0	0	0.0001	0.0001	0	0	0.0005	0	0	0.0001	0.0001
	Min	1.0359	1.0127	1.0005	1.0008	1.0310	1.0148	1.0162	1.0481	1	1.0071	0.9881
	Max	1.1346	1.0962	1.0274	1.0194	1.1451	1.1154	1.0799	1.2604	1.0680	1.0852	1.0400

Table 4 The normalized results of xGP-ACO+2-opt, ACO+2-opt baselines, and LLM-ACO+2-opt on the test datasets

TSP	AS+2-opt				ACS+2-opt				MMAS+2-opt			
	Baseline	LLM	GP	xGP	Baseline	LLM	GP	xGP	Baseline	LLM	GP	xGP
20	Mean	1 –	1 =	1 =	1.0003 +	1 =	1 =	1	1.0002 +	1 –	1.0001 –	1
	Std	0	0	0	0	0	0	0	0	0	0	0
	Min	1	1	1	1	1	1	1	1	1	1	1
	Max	1	1	1.0063	1.0186	1.0000	1.0186	1	1.0186	1	1.0090	1.0103
20–100	Mean	1.0018 +	1.0037 +	1.0008 +	1.0002	1.0021 +	1.0046 +	1.0008 +	1.0007 =	1.0008 +	1.0008 +	1.0007
	Std	0	0	0	0	0	0	0.0001	0	0	0.0001	0.0001
	Min	1	1	1	1	1	1	1	1	1	1	1
	Max	1.0101	1.0162	1.0452	1.0053	1.0097	1.0173	1.0080	1.0102	1.0084	1.0495	1.0728
100	Mean	1.0045 +	1.0091 +	1.0021 +	1.0017	1.0045 +	1.0096 +	1.0017 –	1.0017 +	1.0011 –	1.0013 =	1.0013
	std	0	0	0.0001	0.0001	0	0	0.0001	0	0	0.0001	0.0001
	Min	1	1	1	1	1	1	1	1	1	1	1
	Max	1.0184	1.0269	1.0136	1.0142	1.0211	1.0281	1.0117	1.0112	1.0185	1.0276	1.0276

LLM performed slightly worse than xGP, while the results of xGP-AS-best further narrowed this gap to 5.91%. After introducing the 2-opt local search, the improvements were remarkable, with all gaps reduced to below 1.5%. However, some of the heuristic measures designed by LLM became ineffective and slightly increased the baseline gap instead of reducing it. In contrast, xGP further reduced this gap, with xGP-AS+2-opt-best reducing the gap to 0.7% compared to the optimal solution.

For ACS and xGP-ACS, ACS without local search underperforms due to its heavy reliance on state transition rules. The average gap of 23.02% compared to the optimal solution is unsatisfactory. xGP-ACS reduced the baseline gap to the optimal solution by half, but the heuristic measures designed by LLM performed even better, although the gap of 8.83% compared to the optimal solution remains significant. After introducing the 2-opt algorithm, performance improved remarkably, with all gaps reduced to within 2%. xGP-ACS+2-opt achieved the smallest gap across all instances, halving the gap between the baseline and the optimal solution. While the heuristic measures designed by LLM are highly competitive, they still lag behind xGP.

For MMAS and xGP-MMAS, xGP-MMAS reduced the gap between the baseline and the optimal solution from 9.32% to 3.62%, surpassing the heuristic measures designed by LLM. After introducing the 2-opt algorithm, the baseline gap was halved. The heuristic measures designed by LLM performed remarkably well, with an average gap of 0.37%-an outstanding result. xGP-MMAS+2-opt-best achieved exceptional performance, with an average gap of 0.25%, which is highly satisfactory. In some instances, such as ts225 and pr226, the gap was less than 0.01%. However, the situation with xGP-MMAS is more complex. Although it reduced the gap between the baseline and the optimal solution, the average performance was inferior compared to the version without local search. Closer observation revealed that xGP-MMAS+2-opt achieved a gap of only 0.2% on ch130, and less than 0.5% in instances with fewer than 100 cities. However, as the problem size increased, the gap expanded significantly, often exceeding 5%. This indicates that xGP-MMAS+2-opt is sensitive to both the number of cities and the distribution of the map. In certain independent training runs, overfitting occurred, with the solution performing better only in instances similar in scale to those used in training, leading to an increase in the average gap of xGP-MMAS+2-opt. Consequently, the average gap was higher than that of xGP-MMAS without local search.

Overall, introducing local search can significantly enhance the performance of both ACO and GP-ACO. Although the heuristic measures designed by LLM are impressive, in most cases, the best state transition rules identified by xGP are currently the most effective approach. xGP-MMAS-best can

elevate MMAS to almost the same level as ACO baselines with local search.

After comparing within the same ACO variant, we can now conduct a cross-comparison between different ACO variants and assess the improvements brought by xGP and LLM to each ACO. The heuristic measures designed by LLM are impressive. Although they underperformed compared to the baseline within the AS+2-opt framework, they significantly narrowed the gap between the baseline and the optimal solution in most cases, even surpassing xGP-best within the ACS framework. xGP brought significant improvements to all ACO variants and ACO variants+2-opt, except ACS. Notably, the best state transition rules in xGP, denoted as xGP-best, reduced the gap in the AS baseline by 8.6 percentage points and lowered the gap from the ACS+2-opt baseline's 1.75% to 0.7%. In ACS, which heavily relies on state transition rules and global updates, xGP provides greater enhancements. Within the MMAS+2-opt framework, while xGP-best performed exceptionally well, xGP is prone to overfitting. Nevertheless, xGP significantly improves the performance of all ACO variants except ACS. For instance, xGP reduced the gap in the AS baseline by 8.6 percentage points and lowered the gap from 1.75% to 0.7% in the ACS+2-opt baseline.

Thus, we can response to the Question 5:

- Answer 5: The state transition rules learned by xGP-ACO and xGP-ACO+2-opt demonstrate strong performance across maps of varying scales and distributions. This indicates that the generality capabilities of xGP-ACO extend beyond training scenarios, adapting effectively to diverse and larger-scale environments.

6 Further analysis

6.1 Convergence analysis

To analyze the performance of the state transition rules learned by GP-ACO and xGP-ACO more deeply, we utilize convergence curves to assess their performance. Based on box plots and significance analysis, we depicted only the curves of the ACO with the best state transition rule evolved by GP-ACO and xGP-ACO. We then plotted the convergence curves for the baseline(manually designed) ACO, LLM-ACO, GP-ACO, and xGP-ACO across three scales of TSP problems, as illustrated in the Fig. 7. Additionally, we included convergence curves after the introduction of local search, as shown in Fig. 8. These convergence curves provide a visual representation of how quickly and effectively each algorithm converges, thereby illustrating the performance of the learned state transition rules under different conditions. This comparative analysis highlights the improvements that

Table 5 The results of xGP-ACO, ACO baselines, and LLM-ACO on the TSPLIB datasets

TSP instances	AS				ACS				MMAS			
	Baseline	LLM	xGP	xGP-best	Baseline	LLM	xGP	xGP-best	Baseline	LLM	xGP	xGP-best
bier127	10.75	5.52	6.62	8.83	17.84	6.60	7.83	6.56	6.80	2.06	5.45	0.54
ch130	9.84	5.57	0.88	0.75	20.59	6.19	10.11	9.63	7.98	2.70	0.71	1.53
d493	21.23	11.31	11.20	3.15	32.53	10.90	16.25	13.97	11.22	8.42	4.16	4.98
eil51	7.57	4.01	1.11	0.78	13.20	5.04	10.47	9.29	5.21	1.71	1.11	0.31
fl417	16.90	14.46	4.26	1.81	26.12	15.11	12.68	12.18	9.98	9.29	2.53	4.83
kroA150	13.76	9.26	3.47	2.23	23.55	10.02	13.67	11.62	8.83	4.51	1.92	1.05
kroB100	7.03	5.46	1.35	1.36	16.36	7.41	5.76	6.30	7.43	1.42	1.46	0.26
kroC100	6.92	4.68	1.25	1.08	17.05	5.49	7.75	7.22	7.96	1.14	1.39	0.18
lin318	17.34	10.21	11.25	6.40	26.70	9.65	16.51	15.74	10.73	6.50	3.87	3.93
pr226	15.00	7.28	10.33	9.30	17.58	9.89	12.01	10.27	8.74	2.20	5.62	2.24
pr264	13.62	8.92	8.62	7.52	25.30	9.30	12.75	11.49	9.82	5.99	3.81	3.30
pr299	20.97	10.36	13.36	7.92	33.72	11.21	19.04	17.28	12.23	8.13	4.23	3.69
pr439	23.52	10.94	21.67	16.86	36.07	13.35	19.71	16.19	12.27	7.31	7.53	4.45
rat99	9.00	6.50	1.18	1.01	15.58	8.60	10.54	10.76	8.49	3.61	1.85	1.97
ts225	9.25	3.70	14.82	19.71	23.17	3.74	8.91	5.56	12.08	3.10	8.71	0.70
Avg. opt. gap	13.51	7.88	7.43	5.91	23.02	8.83	12.27	10.93	9.32	4.54	3.62	2.26

Table 6 The results of xGP-ACO+2-opt, ACO+2-opt baselines, and LLM-ACO+2-opt on the TSPLIB datasets

TSP instances	AS+2opt				ACS+2opt				MMAS+2opt			
	Baseline	LLM	xGP	xGP-best	Baseline	LLM	xGP	xGP-best	Baseline	LLM	xGP	xGP-best
bier127	0.85	0.74	0.99	0.51	1.26	0.71	0.38	0.26	3.60	0.07	2.05	0.03
ch130	0.94	1.25	0.03	0.25	1.89	1.40	0.94	1.01	4.00	0.22	0.20	0.14
d493	2.30	2.60	1.82	1.42	3.30	2.25	2.07	1.82	7.96	1.12	4.41	0.58
eil51	0.95	0.90	0.39	0.68	1.50	0.94	0.72	0.71	4.38	0.70	0.41	0.69
fl417	1.13	1.67	0.35	0.83	1.74	1.62	1.07	1.05	2.60	0.59	5.49	0.52
kroA150	1.30	1.81	0.52	0.69	2.04	2.11	0.67	0.56	4.60	0.21	1.41	0.04
kroB100	0.52	0.65	0.27	0.26	0.81	0.40	0.18	0.21	3.06	0.09	0.25	0.08
kroC100	0.28	0.67	0.22	0.008	0.65	0.81	0.04	0.04	2.56	0.05	0.26	0.03
lin318	2.38	2.41	1.26	1.72	3.29	2.06	1.65	1.03	5.74	0.58	5.74	0.45
pr226	0.40	0.86	0.55	0.26	0.89	1.03	0.51	0.52	1.14	0.03	8.63	0.007
pr264	1.04	0.73	0.89	0.21	1.43	2.49	0.27	0.16	5.45	0.007	5.49	0.07
pr299	1.62	1.85	1.66	1.22	2.43	1.96	1.18	1.02	7.60	0.33	6.35	0.18
pr439	1.49	1.84	3.50	1.26	2.20	1.63	1.34	1.00	7.17	0.55	9.36	0.22
rat99	1.04	1.44	0.38	0.71	2.09	1.16	0.74	0.75	7.35	0.72	0.46	0.69
ts225	0.54	0.74	3.29	0.48	0.72	1.37	0.32	0.29	3.34	0.33	7.61	0.002
Avg. opt. gap	1.12	1.34	1.08	0.70	1.75	1.46	0.80	0.70	4.70	0.37	3.87	0.25

xGP-ACO, with or without local search, brings over traditional and modified versions of ACO.

In Fig. 7, the convergence curves of the AS variants initially display the algorithmic performance across varying scales of TSP problems. In TSP20, the xGP-AS algorithm rapidly converges within the initial 10 iterations, ultimately reaching the lowest optimal cost of approximately 1.0. GP-AS also quickly reduces costs in the early stages but gradually

stabilizes after 20 iterations, with a final optimal cost slightly higher than that of xGP-AS. The performance of LLM-AS is slightly inferior to GP-AS, with a slower convergence rate and a final cost marginally higher than GP-AS. The AS baseline shows the slowest convergence, with the best cost reaching about 1.05 after 100 iterations. In TSP20-100, xGP-AS sees its best cost drop to around 1.0 after the first 20 iterations, while GP-AS's final best cost is slightly higher

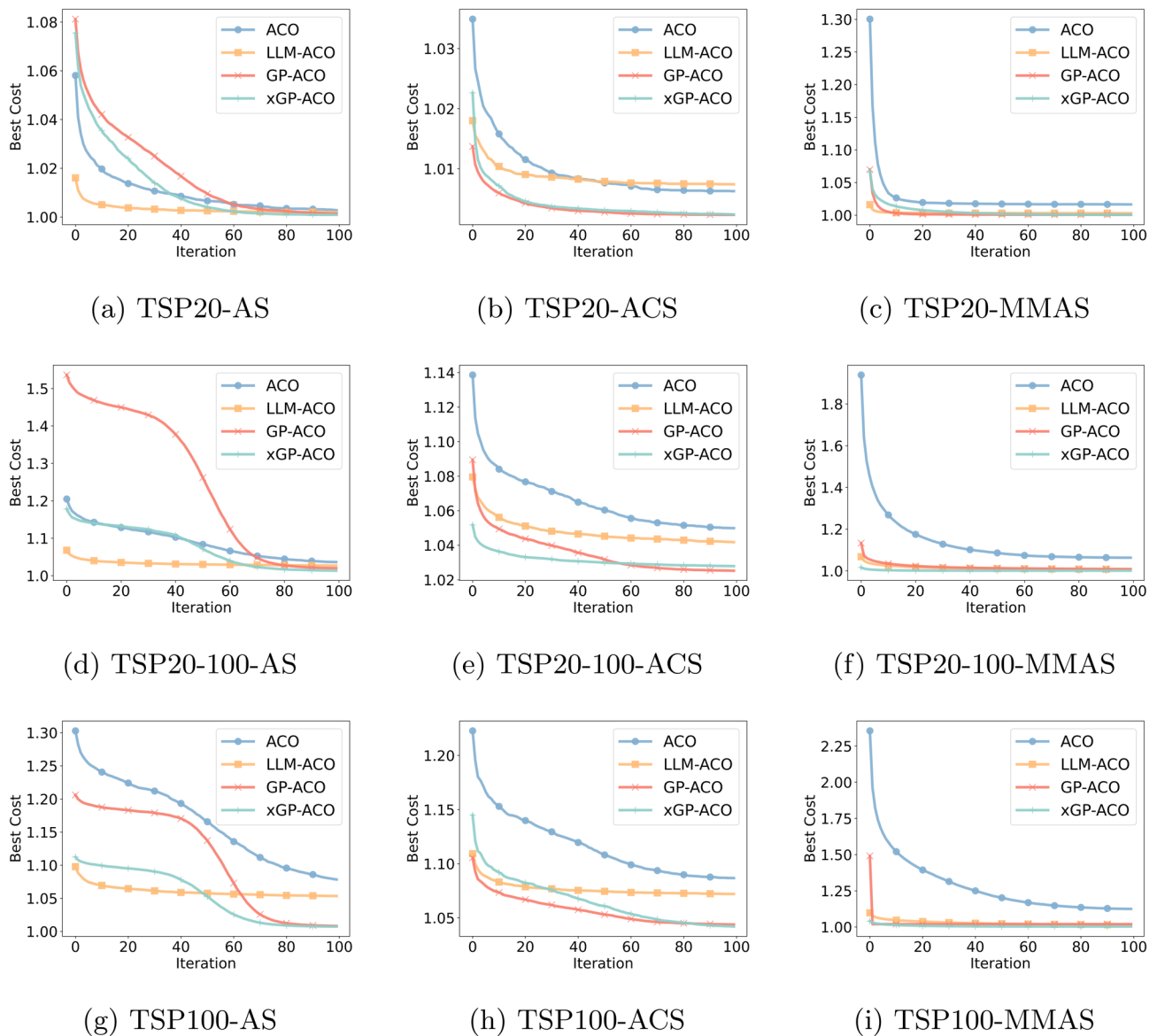


Fig. 7 Convergence of ACO baseline, LLM-ACO, GP-ACO and xGP-ACO

than that of xGP-AS, and LLM-AS's best cost is higher than GP-AS. The AS baseline's best cost remains above 1.1 after 100 iterations. In TSP100, xGP-AS rapidly reduces the best cost within the first 50 iterations, eventually dropping to about 1.05, while GP-AS's best cost is around 1.1, LLM-AS's best cost is approximately 1.12, and the AS baseline's best cost remains above 1.2. Secondly, the convergence curves for the ACS variants also confirm the superiority of xGP-ACS. In TSP20, xGP-ACS's best cost drops below 1.0 after just five iterations, GP-ACS performs closely to xGP-ACS, with its best cost falling to about 1.0 after 20 iterations, while LLM-ACS performs slightly worse, with a best cost just above 1.0. The ACS baseline's best cost is around 1.02 after 100 iterations. In TSP problems ranging from 20 to

100 cities, xGP-ACS's best cost drops below 1.02 within the first 10 iterations and remains stable thereafter, GP-ACS's best cost falls to about 1.04 after 30 iterations, LLM-ACS's best cost is approximately 1.05, and the ACS baseline's best cost is about 1.08 after 100 iterations. In TSP100, xGP-ACS rapidly decreases its best cost to about 1.04 after 20 iterations and gradually reduces it further to 1.02 in subsequent iterations, GP-ACS's best cost falls to about 1.06 after 50 iterations, LLM-ACS's best cost is about 1.07, and the ACS baseline's best cost remains above 1.1. Lastly, the convergence curves of the MMAS variants show that xGP-MMAS performs the best across different scales of TSP problems. In TSP20, xGP-MMAS quickly converges after the initial 10 iterations, with the best cost dropping below 1.0, GP-MMAS

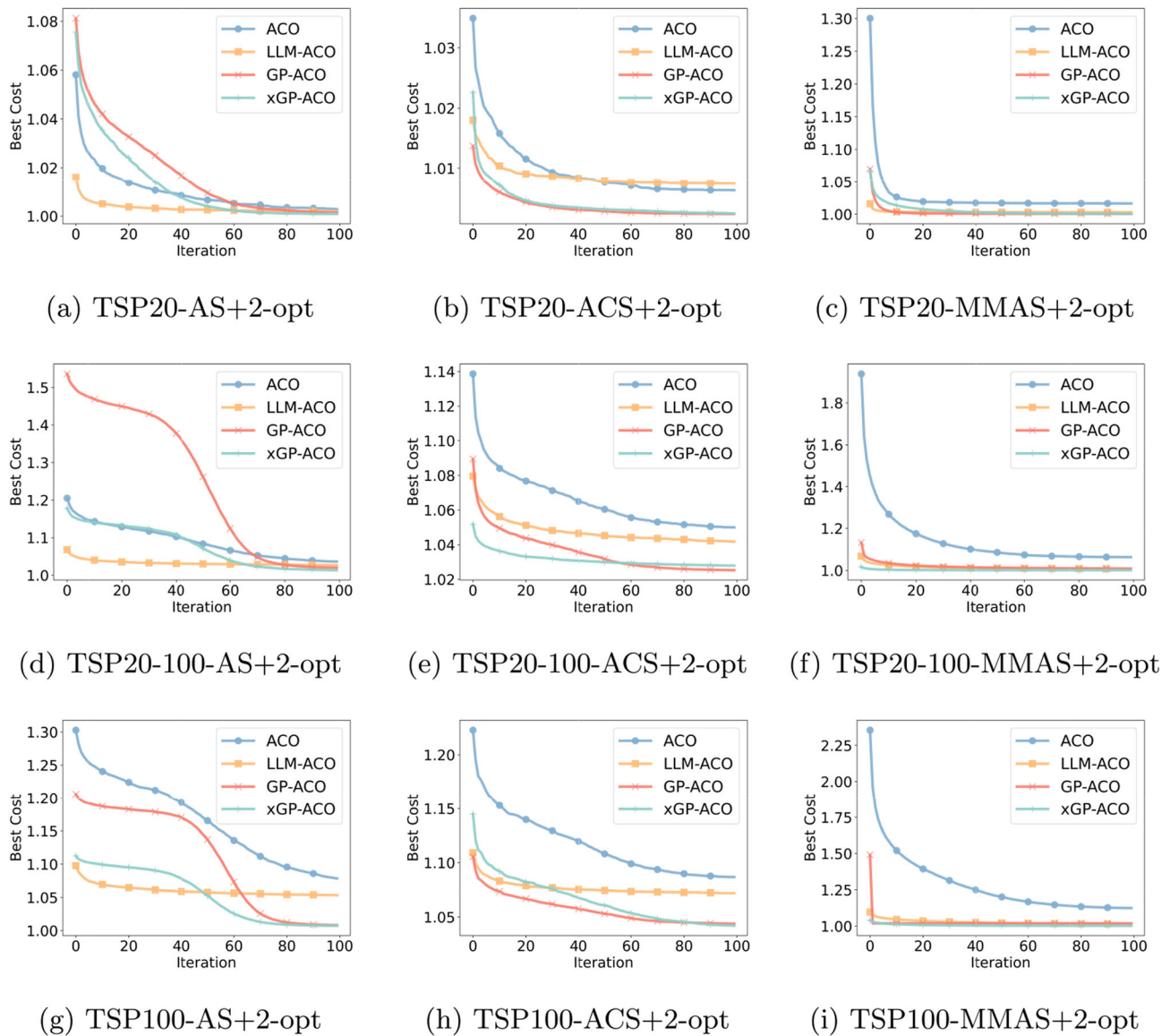


Fig. 8 Convergence of ACO+2-opt baseline, LLM-ACO+2-opt, GP-ACO+2-opt and xGP-ACO+2-opt

performs close to xGP-MMAS, with its best cost falling to about 1.0 after 20 iterations. LLM-MMAS performs slightly worse than GP-MMAS, with a best cost just above 1.0. The MMAS baseline's best cost is around 1.05 after 100 iterations. In TSP20-100, xGP-MMAS rapidly drops to about 1.0 after the initial 10 iterations, GP-MMAS's best cost falls to about 1.02 after 30 iterations, while LLM-MMAS's best cost is approximately 1.03, and the MMAS baseline's best cost is about 1.05 after 100 iterations. In TSP100, the xGP-MMAS algorithm rapidly reduces its best cost to about 1.02 after 20 iterations and gradually lowers it to 1.0 in subsequent iterations, GP-MMAS's best cost falls to about 1.04 after 50 iterations, LLM-MMAS's best cost is about 1.05, and the MMAS baseline's best cost remains above 1.1.

Following the integration of 2-opt local search, as illustrated in Fig. 8, the convergence curves for the AS variants show distinct performance characteristics across various TSP scales. In TSP20, the xGP-AS algorithm quickly converges in the initial iterations, reducing the best cost to approximately 1.0; GP-AS stabilizes after 20 iterations with a slightly higher final best cost than xGP-AS; LLM-AS converges slower with a best cost around 1.0; and the AS baseline achieves a best cost of about 1.035 after 100 iterations. In TSP20-100, xGP-AS's best cost drops to about 1.0 after the first 20 iterations, while GP-AS and LLM-AS have best costs slightly higher than that of xGP-AS, with the AS baseline's best cost still above 1.025. In TSP100, xGP-AS rapidly decreases the best cost to about 1.2 within the first 50 iterations, whereas GP-AS and LLM-AS achieve best costs of 1.4 and 1.3, respectively,

with the AS baseline still exceeding 1.5. For the ACS variants, the convergence curves reveal that in TSP20, xGP-ACS's best cost falls below 1.0 after just five iterations, with GP-ACS and LLM-ACS showing similar performance, slightly above 1.0, and the ACS baseline reaching about 0.0006 after 100 iterations. In TSP20-100, xGP-ACS's best cost drops below 1.02 within the first 10 iterations and remains stable, while GP-ACS and LLM-ACS achieve best costs of 1.04 and 1.05, respectively, with the ACS baseline at about 1.08. In TSP100, xGP-ACS quickly reduces its best cost to 1.04 within the first 20 iterations and further lowers it to 1.02, whereas GP-ACS and LLM-ACS have best costs of 1.06 and 1.07, respectively, with the ACS baseline still above 1.1. Lastly, the convergence curves for the MMAS variants indicate that in TSP20, xGP-MMAS quickly converges after the first 10 iterations, dropping the best cost below 1.0, with GP-MMAS and LLM-MMAS performing similarly, slightly above 1.0, and the MMAS baseline achieving about 1.05. In TSP20-100, xGP-MMAS's best cost rapidly falls to 1.0 after the initial 10 iterations, while GP-MMAS and LLM-MMAS reach best costs of 1.02 and 1.03, respectively, and the MMAS baseline is around 1.05. In TSP100, xGP-MMAS reduces its best cost to 1.02 within the first 20 iterations and gradually decreases it to 1.0, with GP-MMAS and LLM-MMAS achieving best costs of 1.04 and 1.05, respectively, while the MMAS baseline remains above 1.1.

In summary, regardless of whether local search is employed, and across all variants-AS, ACS, and MMAS-the xGP-ACO algorithm consistently exhibits superior performance, demonstrating the fastest convergence speeds and the lowest optimal costs in all tested scales of TSP problems. In contrast, the traditional ACO baseline performs poorly in all scenarios, characterized by slower convergence and significantly higher final optimal costs compared to other variants. The GP-ACO algorithm generally performs well, slightly inferior to xGP-ACO but significantly better than LLM-ACO. LLM-ACO displays consistent results across all scenarios, but its convergence speed and final optimal costs are modestly higher than those of GP-ACO. These findings underscore the distinct advantage of the xGP-ACO algorithm in solving TSP problems of varying scales, particularly pronounced in larger-scale problems.

6.2 Evolved state transition rules analysis

To answer Question 6, we need to analyse the evolved state transition rules of GP-ACO and xGP-ACO.

Based on the box plots and significance analysis, we identified several top-performing GP trees in GP-ACO and analyzed them as heuristic forms, as shown in Table 7. In the best tree found by GP-AS, we observe that the power of d in the denominator is significantly high, whereas the power of φ in the numerator is not as pronounced. In GP-ACS,

d often appears as an integral part, usually preceded by a negative sign. In GP-MMAS, the power of d in the denominator is lower compared to GP-ACS, and there are instances where the heuristic values are consistently negative. The scenarios in GP-AS and GP-MMAS are similar, typically featuring complex denominators and simple numerators, while the situation in GP-ACS is distinctly different, with the best trees of GP-ACS primarily comprising integral parts. This is attributed to the fact that GP-AS and GP-MMAS predominantly rely on a roulette-wheel selection mechanism, whereas GP-ACS, in most cases, directly chooses based on the highest heuristic value. The magnitude of the values calculated by the integral parts results in the greatest differentiation, enabling such individuals to stand out within the population.

Additionally, we investigated the best-performing state transition rules of xGP-ACO identified in TSP100, as detailed in Table 8. The associated box plots and convergence curves for xGP-ACO, illustrating that xGP-ACO effectively utilized the new terminals $\tilde{d}$ and $\tilde{\varphi}$. Specifically, xGP-MMAS employed only the new terminal $\tilde{\varphi}$, while xGP-ACS used solely $\tilde{d}$, and xGP-AS included both $\tilde{d}$ and n . Notably, the terminal n_u was not used in any of the best state transition rules.

The state transition rule of xGP-ACS+2-opt-best is remarkably streamlined yet effective, incorporating only $\tilde{d}$, and achieving excellent results with this succinct expression. In contrast, xGP-AS+2-opt-best also includes n alongside $\tilde{d}$. Although its expression is less concise, its performance closely mirrors that of xGP-ACS+2-opt-best. In the case of xGP-MMAS+2-opt-best, distance-related terminals are entirely omitted, relying solely on pheromone and node count, which leads to results significantly surpassing those of other ACO and GP-ACO variants. This highlights the critical role of pheromone and global pheromone information in enhancing the effectiveness of the MMAS strategy.

Now we can answer the Question 6:

- Answer 6: GP-ACO and xGP-ACO exhibit excellent interpretability.

7 Conclusions and future work

This paper conducts an in-depth study of the GP-ACO algorithm, which utilizes GP to automatically design the state transition rules within ACO algorithms. The primary goals of this paper are to answer four research questions that were formulated concerning generality, the impact of different ACO variants on GP-ACO, the influence of the 2-opt local search on GP-ACO, and the effects of incorporating more global information into GP-ACO. To address these questions, we designed four progressive experiments, each aimed at exploring one of these specific issues. From the experi-

Table 7 New State Transition rules learned by GP-ACO

Algorithm	TSP20	TSP20-100	TSP100
GP-AS	$\frac{\varphi}{(d-2\varphi)d^2} - \varphi$	$\frac{\varphi}{(\varphi+d^4)d^5}$	$\frac{2\varphi}{(2\varphi+d^2)(2\varphi+d^4)d^3}$
GP-ACS	$\varphi d^3 - \frac{d^3}{\varphi} + 1$	$\frac{(d^3-1)\varphi}{(\varphi-d)d^4} - d$	$\frac{2\varphi}{(\varphi+d)^2d} - 2d^2 + 1$
GP-MMAS	$-\varphi d - \frac{2d^2}{\varphi+d}$	$-\frac{2\varphi}{(\varphi+\varphi d+2d^2)d}$	$\frac{\varphi+\varphi(\varphi+d)d^2}{(\varphi+d)^2d^4}$

Table 8 New State Transition rules learned by xGP-ACO

Local Search	xGP-AS	xGP-ACS	xGP-MMAS
None	$\frac{n^2\varphi^2\bar{d}^2}{\varphi d\bar{d}+d^2}$	$\left(\frac{\bar{d}}{d}\right)^3 - \frac{\varphi}{d^2}$	$\frac{2\bar{\varphi}+\varphi d}{\bar{\varphi}^4+d^8}$
2-opt	$\frac{n^3\varphi^3\bar{d}^2}{(n\varphi+1)(n+\varphi)d}$	$\frac{\varphi^2\bar{d}}{d^2}$	$-\varphi^2 + (n - n_u - 2)\varphi - n_u - \bar{\varphi}$

mental results, we observe that: 1. GP-ACO exhibits robust generality and performs well across problems of different scales and distributions; 2. Different ACO variants do not affect the learning capability of GP-ACO; 3. The introduction of 2-opt local search diminishes the learning capabilities of GP-ACO in the MMAS+2-opt variant, which has a reduced dependency on state transition rules; 4. Incorporating additional global information (xGP-ACO) significantly enhances the learning capabilities of GP-ACO; 5. GP-ACO and xGP-ACO show excellent interpretability.

Future directions could involve increasing the scale of training and testing, and applying the approach to other combinatorial optimization problems. Furthermore, this research highlights the potential of designing terminals that incorporate global information to enhance the performance of state transition rules. Future studies could employ learning-based methods to automatically extract features from graphs as global information for GP-ACO input. This approach would likely be particularly potent in problems with more graph information, such as VRP with time window (VRPTW) and EVRP. Moreover, designing effective surrogate methods to accelerate the evaluation process of GP becomes even more crucial when ACO is nested within GP.

Acknowledgements The work of Bo-Cheng Lin was supported by the China Scholarship Council (CSC)/Victoria University Scholarship. This work is partially supported by an MBIE Endeavour Smart Idea Fund.

Funding Open Access funding enabled and organized by CAUL and its Member Institutions.

Open Access This article is licensed under a Creative Commons Attribution 4.0 International License, which permits use, sharing, adaptation, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if changes were made. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copy-

right holder. To view a copy of this licence, visit <http://creativecommons.org/licenses/by/4.0/>.

References

- Dorigo M, Gambardella LM (1997) Ant colony system: a cooperative learning approach to the traveling salesman problem. *IEEE Trans Evol Comput* 1(1):53–66
- Dorigo M, Stützle T (2019) Ant colony optimization: overview and recent advances. In: Gendreau M, Potvin J-Y (eds) *Handbook of metaheuristics*. international series in operations research & management science. Springer, Cham, pp 311–351
- Ye H, Wang J, Cao Z, Liang H, Li Y (2023) DeepACO: neural-enhanced Ant Systems for Combinatorial Optimization. *arXiv*
- Stützle T, Hoos HH (2000) MAX-MIN ant system. *Futur Gener Comput Syst* 16(8):889–914
- Runka A (2009) Evolving an edge selection formula for ant colony optimization. In: *Proceedings of the 11th annual conference on genetic and evolutionary computation*. GECCO '09. Association for Computing Machinery, New York, NY, USA, pp 1075–1082
- Sun Y, Wang S, Shen Y, Li X, Ernst AT, Kirley M (2022) Boosting ant colony optimization via solution prediction and machine learning. *Comput Oper Res* 143:105769
- Vaswani A, Shazeer N, Parmar N, Uszkoreit J, Jones L, Gomez AN, Kaiser Ł, Polosukhin I (2017) Attention is all you need. In: *Advances in neural information processing systems*, vol 30
- Radford A, Narasimhan K, Salimans T, Sutskever I. Improving language understanding by generative pre-training
- Liu S, Chen C, Qu X, Tang K, Ong Y-S (2023) Large language models as evolutionary optimizers. *arXiv*
- Ye H, Wang J, Cao Z, Song G (2024) ReEvo: large language models as hyper-heuristics with reflective evolution. *arXiv*
- Wu X, Wu S-h, Wu J, Feng L, Tan KC (2024) Evolutionary computation in the era of large language model: survey and roadmap. *arXiv*
- Tavares J, Pereira FB (2010) Evolving strategies for updating pheromone trails: a case study with the TSP. In: Schaefer R, Cotta C, Kołodziej J, Rudolph G (eds) *Parallel problem solving from nature, PPSN XI*. Springer, Berlin, Heidelberg, pp 523–532
- Tavares J, Pereira FB (2011) Designing pheromone update strategies with strongly typed genetic programming. In: Silva S, Foster JA, Nicolau M, Machado P, Giacobini M (eds) *Genetic programming*. Springer, Berlin, Heidelberg, pp 85–96
- Yang C, Wang X, Lu Y, Liu H, Le QV, Zhou D, Chen X (2023) Large language models as optimizers. *arXiv*

15. Liu F, Tong X, Yuan M, Lin X, Luo F, Wang Z, Lu Z, Zhang Q (2024) An example of evolutionary computation + large language model beating human: design of efficient guided local search. *arXiv*
16. Chen A, Dohan DM, So DR (2023) EvoPrompting: language models for code-level neural architecture search. *arXiv*
17. Linardatos P, Papastefanopoulos V, Kotsiantis S (2021) Explainable AI: a review of machine learning interpretability methods. *Entropy* 23(1):18
18. Mei Y, Chen Q, Lensen A, Xue B, Zhang M (2023) Explainable artificial intelligence by genetic programming: a survey. *IEEE Trans Evol Comput* 27(3):621–641
19. Poli R, Langdon WB, McPhee NF, Koza JR (2008) A field guide to genetic programming. Lulu Press, Morrisville, NC
20. Zhang F, Mei Y, Nguyen S, Zhang M (2023) Survey on genetic programming and machine learning techniques for heuristic design in job shop scheduling. *IEEE Trans Evol Comput* 1
21. Dorigo M, Maniezzo V, Colomi A (1991) The ant system: an auto-catalytic optimizing process
22. Dorigo M, Birattari M, Stutzle T (2006) Ant colony optimization. *IEEE Comput Intell Mag* 1(4):28–39
23. Dinh QT, Do DD, Hà MH (2021) Ants can solve the parallel drone scheduling traveling salesman problem. In: *Proceedings of the genetic and evolutionary computation conference. GECCO '21*. Association for Computing Machinery, New York, NY, USA, pp 14–21
24. Akhtar A (2019) Evolution of ant colony optimization algorithm: a brief literature review. *arXiv*
25. Jia Y-H, Mei Y, Zhang M (2022) Confidence-based ant colony optimization for capacitated electric vehicle routing problem with comparison of different encoding schemes. *IEEE Trans Evol Comput* 26(6):1394–1408
26. Montemanni R, Gambardella LM, Rizzoli AE, Donati AV (2005) Ant colony system for a dynamic vehicle routing problem. *J Comb Optim* 10(4):327–343
27. Shmygelska A, Hoos HH (2005) An ant colony optimisation algorithm for the 2D and 3D hydrophobic polar protein folding problem. *BMC Bioinform* 6(1):30
28. Sánchez M, Cruz-Duarte JM, Ortíz-Bayliss J, Ceballos H, Terashima-Marin H, Amaya I (2020) A systematic review of hyper-heuristics on combinatorial optimization problems. *IEEE Access* 8:128068–128095
29. Burke EK, Gendreau M, Hyde M, Kendall G, Ochoa G, Özcan E, Qu R (2013) Hyper-heuristics: a survey of the state of the art. *J Oper Res Soc* 64(12):1695–1724
30. Qu R, Burke EK, McCollum B, Merlot LTG, Lee SY (2009) A survey of search methodologies and automated system development for examination timetabling. *J Sched* 12(1):55–89
31. Mei Y, Zhang M (2016) A comprehensive analysis on reusability of GP-evolved job shop dispatching rules. In: *2016 IEEE congress on evolutionary computation (CEC)*, pp 3590–3597
32. MacLachlan J, Mei Y, Branke J, Zhang M (2020) Genetic programming hyper-heuristics with vehicle collaboration for uncertain capacitated arc routing problems. *Evol Comput* 28(4):563–593
33. Ochoa G, Qu R, Burke EK (2009) Analyzing the landscape of a graph based hyper-heuristic for timetabling problems. In: *Proceedings of the 11th annual conference on genetic and evolutionary computation. GECCO '09*. Association for Computing Machinery, New York, NY, USA, pp 341–348
34. Fan H, Xiong H, Goh M (2021) Genetic programming-based hyper-heuristic approach for solving dynamic job shop scheduling problem with extended technical precedence constraints. *Comput Oper Res* 134:105401
35. Burke EK, Hyde MR, Kendall G, Ochoa G, Ozcan E, Woodward JR (2009) Exploring hyper-heuristic methodologies with genetic programming. In: *Mumford CL, Jain LC (eds) Computational intelligence: collaboration, fusion and emergence*. Intelligent Systems Reference Library, pp 177–201. Springer, Berlin, Heidelberg
36. Xu M, Mei Y, Zhang F, Zhang M (2024) Genetic programming and reinforcement learning on learning heuristics for dynamic scheduling: a preliminary comparison. *IEEE Comput Intell Mag* 19(2):18–33
37. Lopez-Ibanez M, Stutzle T (2012) The automatic design of multi-objective ant colony optimization algorithms. *IEEE Trans Evol Comput* 16(6):861–875
38. Aziz ZA (2015) Ant colony hyper-heuristics for travelling salesman problem. *Procedia Comput Sci* 76:534–538
39. Tiwari PK, Vidyarthi DP (2016) Improved auto control ant colony optimization using lazy ant approach for grid scheduling problem. *Futur Gener Comput Syst* 60:78–89
40. Tavares J, Pereira FB (2012) Automatic design of ant algorithms with grammatical evolution. In: *Moraglio A, Silva S, Krawiec K, Machado P, Cotta C (eds) Genetic programming. Lecture notes in computer science*. Springer, Berlin, Heidelberg, pp 206–217
41. Xi Z, Chen W, Guo X, He W, Ding Y, Hong B, Zhang M, Wang J, Jin S, Zhou E, Zheng R, Fan X, Wang X, Xiong L, Zhou Y, Wang W, Jiang C, Zou Y, Liu X, Yin Z, Dou S, Weng R, Cheng W, Zhang Q, Qin W, Zheng Y, Qiu X, Huang X, Gui T (2023) The rise and potential of large language model based agents: a survey. *arXiv*
42. Pluhacek M, Kazikova A, Kadavy T, Viktorin A, Senkerik R (2023) Leveraging large language models for the generation of novel metaheuristic optimization algorithms. In: *Proceedings of the companion conference on genetic and evolutionary computation. GECCO '23 Companion*. Association for Computing Machinery, New York, NY, USA, pp 1812–1820
43. Liu F, Tong X, Yuan M, Zhang Q (2023) Algorithm evolution using large language model. *arXiv*
44. Liu S, Zhang Y, Tang K, Yao X (2023) How good is neural combinatorial optimization: a systematic evaluation on the traveling salesman problem. *IEEE Comput Intell Mag* 18(3):14–28
45. Liu Y, Qiu J, Hart E, Yu Y, Gan Z, Li W (2023) Learning-based neural ant colony optimization. In: *Proceedings of the genetic and evolutionary computation conference. GECCO '23*. Association for Computing Machinery, New York, NY, USA, pp 47–55
46. Sanchez-Lengeling B, Reif E, Pearce A, Wiltchko AB (2021) A gentle introduction to graph neural networks. *Distill* 6(9):33
47. Sun M, Zhou K, He X, Wang Y, Wang X (2022) GPPT: graph pre-training and prompt tuning to generalize graph neural networks. In: *Proceedings of the 28th ACM SIGKDD conference on knowledge discovery and data mining. KDD '22*. Association for Computing Machinery, New York, NY, USA, pp 1717–1727
48. Cook WJ, Applegate DL, Bixby RE, Chvátal V (2011) The traveling salesman problem: a computational study. Princeton University Press, Princeton
49. Fischetti M, Salazar-Gonzalez J-J, Toth P (2007) The generalized traveling salesman and orienteering problems. In: *Du D-Z, Pardalos PM, Gutin G, Punnen AP (eds) The traveling salesman problem and its variations*, vol 12. Springer, Boston, MA, pp 609–662
50. Reinelt G (1991) TSPLIB—a traveling salesman problem library. *ORSA Journal on Computing*

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.